东北大学 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY



Programming in Python

—— PART I Basics

雨课堂 Rain Classroom

Chapter 4: Functions and Modules

Contents

1. Functions
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. Lambda expressions
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. Modules
6. Game example

雨课堂 Rain Classroom

Defining a Function

- Here's a simple function named greet_user() that prints a greeting:

```
# Example-1:
def greet_user():
    """Display a simple greeting."""
    print("Hello!")

greet_user()
```

- 在Python中，函数声明和函数定义是视为一体的
- 在Python中不用指定返回值的类型。
- 函数参数数量任意，也不用指定类型（Python中变量都是弱类型的，Python会根据值来维护变量的类型）
- return可选，可出现在函数体任何位置。如果没有return语句，会自动返回None（空值），如果有return语句，但后面没接任何表达式或值，则也返回None。

- This is the basic syntax for a function definition

```
def  <name> (<formal parameters>):
    the body of the function
    return <return expression>        # Non-essential
```

Defining a Function

- Example:

```
def printHello():  # print the 'hello' string
  print ('hello')
```

```
def printNum():   # Output 0--9 numbers
  for i in range(0,10):
    print (i)
  return
```

```
def add(a,b):      # Realize the sum of two numbers
    return a+b
```

Calling a Function

- Example:

```
def add(a,b):
    return a+b
print ( add(1,2) )
```

- Calling the function before the function definition is not allowed in Python

```
print (add(1,2))
def add(a,b):
    return a+b
Traceback (most recent call last):
  File "<ipython-input-1-7184cfcb2d2f>", line 1, in <module>
    print ( add(1,2) )
NameError: name 'add' is not defined
```


雨课堂 Rain Classroom

Calling a Function

- Example: Find the greatest common divisor

```
def hcf(x, y):      # 定义一个函数，该函数返回两个数的最大公约数
  if x > y:
    smaller = y
  else:
    smaller = x
  for i in range(1, smaller + 1):
    if((x % i == 0) and (y % i == 0)):          # x, y同时整除 i，则 i是最大公约数
      hcf = i
  return hcf

num1 = int(input("输入第一个数字: "))
num2 = int(input("输入第二个数字: "))
print(f"{num1} 和 {num2} 的最大公约数为: {hcf(num1, num2)}")    # 函数调用
```

Calling a Function

- Example: Find the greatest common divisor

```
def hcf(x, y):      # 定义一个函数，该函数返回两个数的最大公约数
  if x > y:
    smaller = y
  else:
    smaller = x
  for i in range(smaller, 0, -1):
    if((x % i == 0) and (y % i == 0)):          # x, y同时整除 i，则 i是最大公约数
      hcf = i
      return hcf

num1 = int(input("输入第一个数字: "))
num2 = int(input("输入第二个数字: "))
print(f"{num1} 和 {num2} 的最大公约数为: {hcf(num1, num2)}")    # 函数调用
```

Chapter 4: Functions and Modules

Contents

1 Functions
- 1 Defining a Function and Using it
- 2 **Passing Information to a Function**
- 3 Python decorators
- 4 Lambda expressions
- 5 Return Values

2 The scope of a variable

3 Closures and recursive calls to functions

4 Build-in Function

5 Modules

6 Game example

Passing Information to a Function

- When we learn functions of Python language , the main problems encountered are:
  - Arguments and Parameters (实参和形参)
  - Passing Arguments (传递实参)
  - Return Values (返回值)
  - variable scope (变量作用域)

Arguments and Parameters (实参和形参)

- **Parameters:** Variables in parentheses after the function name when defining a function.
- Example:

```
def add(a,b):          # Here a and b are the parameters
    return a+b
```

- **Arguments**: a piece of information that's passed from a function call to a function.
- Example:

```
add(1, 2)      # Here 1 and 2 are the arguments
x=2
y = 2
add(x, y)      # Here x and y are the arguments
```

Passing Arguments (传递实参)

- Because a function definition can have multiple parameters, a function call may need multiple arguments. You can pass arguments to your functions in a number of ways. You can use:
  - **Default Values**, When writing a function, you can define a default value for each parameter;
  - **positional arguments**, which need to be in the same order the parameters were written;
  - **keyword arguments**, where each argument consists of a variable name and a value;
  - **lists and dictionaries** of values.

- 在C语言中，调用参数时必须依照函数定义时的参数个数以及类型来传递参数，否则就会发生错误。
- 在Python中函数参数定义和传递的方式相比而言就灵活多。

雨课堂 Rain Classroom

Passing Arguments (传递实参)

1. Default Values

```
def display(a='hello', b='wolrd'):        # Default values for each parameter are provided
    print (a+b)

# main program
display()                  # Output: hellowolrd
display(b='world')         # Output: helloworld
display(a='hello')         # Output: hellowolrd
display('world')           # Output: worldwolrd
```

- 函数定义的时候，分别给出了2个参数a和b的默认值
- 函数调用的时候，如果不给函数传递参数，就分别采用默认值，如果给了参数，但是没指明参数名，则从左向右匹配。
- 使用默认值可简化函数调用，还可清楚地指出函数的典型用法。

Passing Arguments (传递实参)

2. Positional Arguments

```
def display(a, b):                    # No default value provided for parameter
    print (a, b)
```

```
# Example 1
display('hello', 'world')
Output:  hello world
```

```
# Example 2
display('world', 'hello')
Output:  world hello
```

```
# Example 3
display('hello')
Output: TypeError: display() missing 1 required positional argument: 'b'
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

3. Keyword Arguments

```
def display(a, b):                    # No default value provided for parameter
    print (a, b)
```

```
# Example
# 避免用户需要牢记位置参数顺序，可以使用关键字传参
display(a='world', b='hello')
display(b='hello', a='world')
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

4. Equivalent Function Calls

- Because positional arguments, keyword arguments, and default values can all be used together, often you'll have several equivalent ways to call a function.

```
def describe_pet(pet_name, animal_type = 'dog'):
    """Display information about a pet."""
    print(f"\nI have a {animal_type}.")
    print(f"My {animal_type}'s name is {pet_name.title()}.")
# A dog named Willie.
describe_pet('willie')
describe_pet(pet_name='willie')
# A hamster(仓鼠) named Harry.
describe_pet('harry', 'hamster')
describe_pet(pet_name='harry', animal_type='hamster')
describe_pet(animal_type='hamster', pet_name='harry')
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

5. Passing an Arbitrary(任意) Number of Arguments

- Sometimes you **won't know** ahead of time **how many arguments** a function needs to accept.
- For example, consider a function that builds a pizza. It needs to accept a number of toppings(配料), but you can't know ahead of time how many toppings a person will want.

将没有匹配的值都放在同一个**元组**中

```
def make_pizza(*toppings):
    """Print the list of toppings that have been requested."""
    print(toppings)
make_pizza('pepperoni')
make_pizza('mushrooms', 'green peppers', 'extra cheese')
```

Passing Arguments (传递实参)

5. Passing an Arbitrary(任意) Number of Arguments

```
def storename(name,*nickName):
    print ('real name is %s' % name)
    if nickName:
        print('小名是', end=' ')
    for nickname in nickName:
        print (nickname, end=' ')
    print('')

# main
storename('张海')
storename('张海', '小海')
storename('张海', '小海', '小豆豆')
```

- 如果要让函数接受不同类型的实参，必须在函数定义中将接纳任意数量实参的形参放在最后。
- Python先匹配位置实参和关键字实参，再将余下的实参都收集到最后一个形参中。

Output Result:

```
real name is 张海

real name is 张海
小名是 小海
real name is 张海
小名是 小海 小豆豆
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

5. Passing an Arbitrary(任意) Number of Arguments

➢'*' and '**' both indicate that 0 to any number of parameters can be accepted

将没有匹配的值都放在一个**元组**中

```
def demo(*p):
    for item in p:
        print (item, end=' ')

# main
demo(1, 2, 3)
# Output Result:
1 2 3
```

将没有匹配的值都放在一个**字典**中

```
def demo(**p):
    for item in p.items():
        print (item, end=' ')

# main
demo(x=1, y=2, z=3)
# Output Result:
('x', 1) ('y', 2) ('z', 3)
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

6. Passing a List

```
def greet_users(names):
    """Print a simple greeting to each user in the list."""
    for name in names:
        msg = f"Hello, {name.title()}!"
        print(msg)

# main
usernames = ['hannah', 'ty', 'margot']
greet_users(usernames)
# Output Result:
Hello, Hannah!
Hello, Ty!
Hello, Margot!
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

Modifying a List in a Function

- Parameter passing in Python is passed ***by value***, which is somewhat similar to the C language.
- Modifying the value of a **parameter** inside a function will not affect the **argument**.
- Example:

```
def addOne(a):
    a += 1
    print(a)

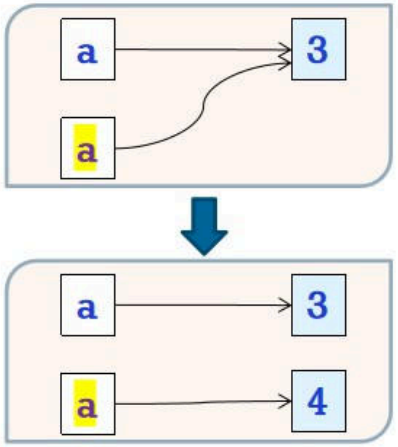
a = 3
addOne(a)  # Output 4
print(a)   # Output 3
```

```
a = 3

addOne(a)

a += 1
print(a)
# Output 4

print(a)
# Output 3
```



雨课堂 Rain Classroom

Passing Arguments (传递实参)

Modifying a List in a Function

```
def modify1(m, K):
    m = 2
    K = [4, 5, 6]
    return
```

```
def modify2(m, K):
    m = 2
    K[0] = 0
    return
```

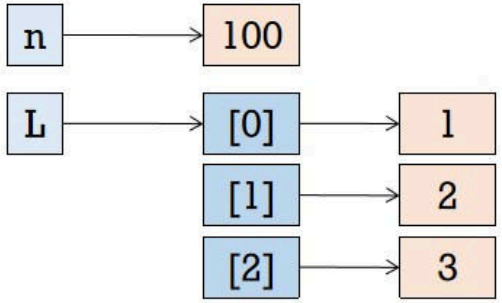
```
# main
n = 100
L = [1,2,3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

Modifying a List in a Function

```
# main
n = 100
L = [1, 2, 3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```

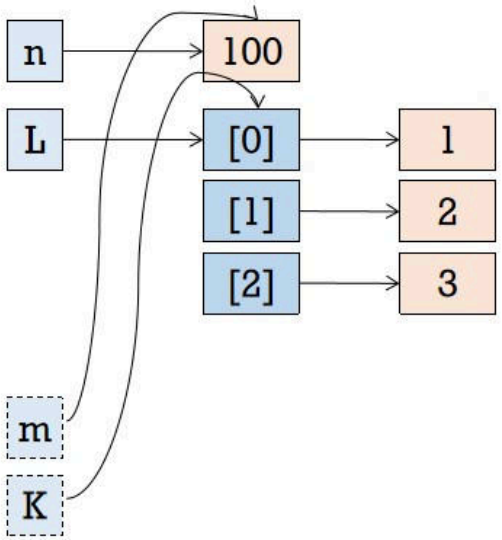


Passing Arguments (传递实参)

Modifying a List in a Function

```
# main
n = 100
L = [1, 2, 3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```

```
def modify1(m, K):
    m = 2
    K = [4, 5, 6]
    return
```

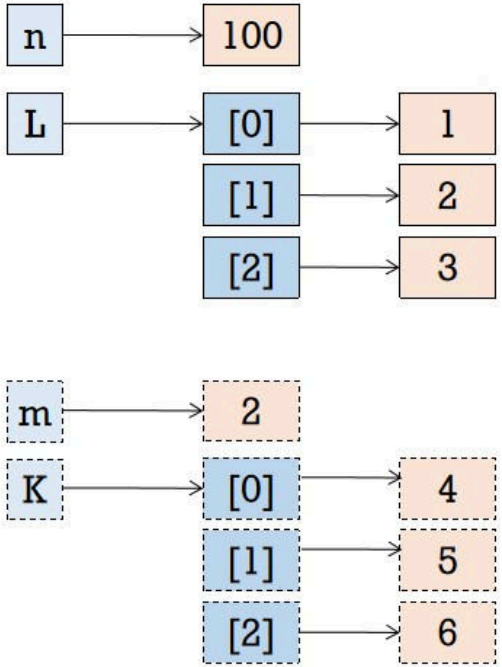


Passing Arguments (传递实参)

Modifying a List in a Function

```
# main
n = 100
L = [1, 2, 3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```

```
def modify1(m, K):
    m = 2
    K = [4, 5, 6]
    return
```

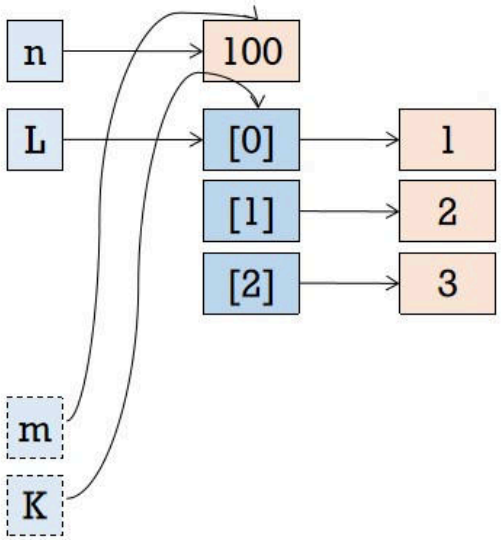


Passing Arguments (传递实参)

Modifying a List in a Function

```
# main
n = 100
L = [1, 2, 3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```

```
def modify2(m, K):
    m = 2
    K[0] = 0
    return
```

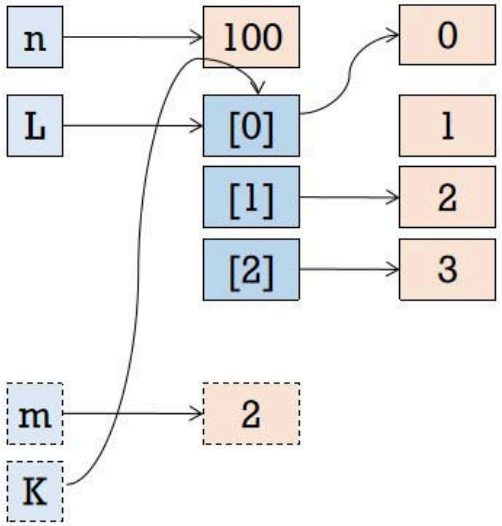


Passing Arguments (传递实参)

Modifying a List in a Function

```
# main
n = 100
L = [1, 2, 3]
modify1(n, L)
print (n, L)        # Output:  100 [1, 2, 3]
modify2(n, L)
print (n, L)        # Output:  100 [0, 2, 3]
```

```
def modify2(m, K):
    m = 2
    K[0] = 0
    return
```



Passing Arguments (传递实参)

- Other Examples: Modifying arguments inside a function

```
def modify(v, item):
    v.append(item)

# main
a = [2]
print(a)
modify(a, 3)
print(a)
```

Output Result:
[2]
[2, 3]

```
def modify(d):
    d['age'] = 38

# main
a = {'name':'Dong', 'age':37, 'sex':'Male'}
print(a)
modify(a)
print(a)
```

Output Result:
{'name':'Dong', 'age':37, 'sex':'Male'}
{'name':'Dong', 'age':38, 'sex':'Male'}

- A mutable(可变的) object can modify the arguments value, while an immutable(不可变的) object cannot modify the arguments value.

Everything is an "object" in Python

```
def hi(name="yasoob"):
    return "hi " + name
print(hi())
Output Result:
hi yasoob

greet = hi    # You can even assign a function to a variable
print(greet())
Output Result:
'hi yasoob'
```

■ 这里没有使用小括号，因为并不是在调用hi函数，而是将它放在greet变量里头。

```
del hi
print(hi())          # Output Result: NameError
print(greet())       # Output Result: 'hi yasoob'
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

7. Passing a function as an argument to another function

- Higher-order function(高阶函数):
  - A function that takes a function as an argument value or
  - A function that returns a function as a return value

```
def apply_twice(f, x):
    return f(f(x))

def square(x):
    return x*x
# main
result = apply_twice(square, 2)
```

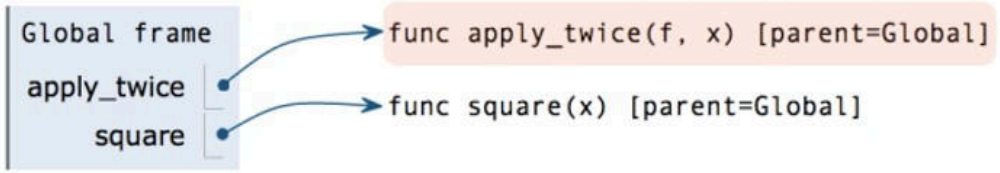
Passing Arguments (传递实参)

7. Passing a function as an argument to another function

- **Higher-order function**(高阶函数):
  - A function that takes a function as an argument value or
  - A function that returns a function as a return value

Names can be Bound to Functional Arguments

```
1 def apply_twice(f, x):
2     return f(f(x))
3
4 def square(x):
5     return x * x
6
7 result = apply_twice(square, 2)
```



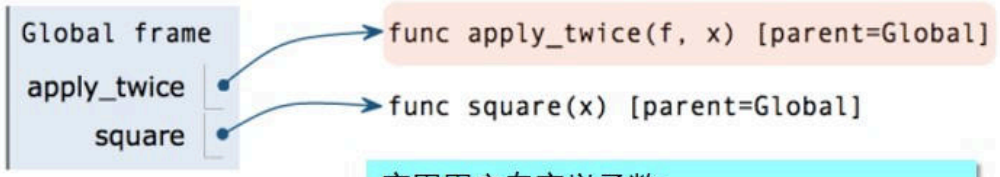
Passing Arguments (传递实参)

7. Passing a function as an argument to another function

- **Higher-order function**(高阶函数):
  - A function that takes a function as an argument value or
  - A function that returns a function as a return value

Names can be Bound to Functional Arguments

```
1  def apply_twice(f, x):
2      return f(f(x))
3
4  def square(x):
5      return x * x
6
7  result = apply_twice(square, 2)
```

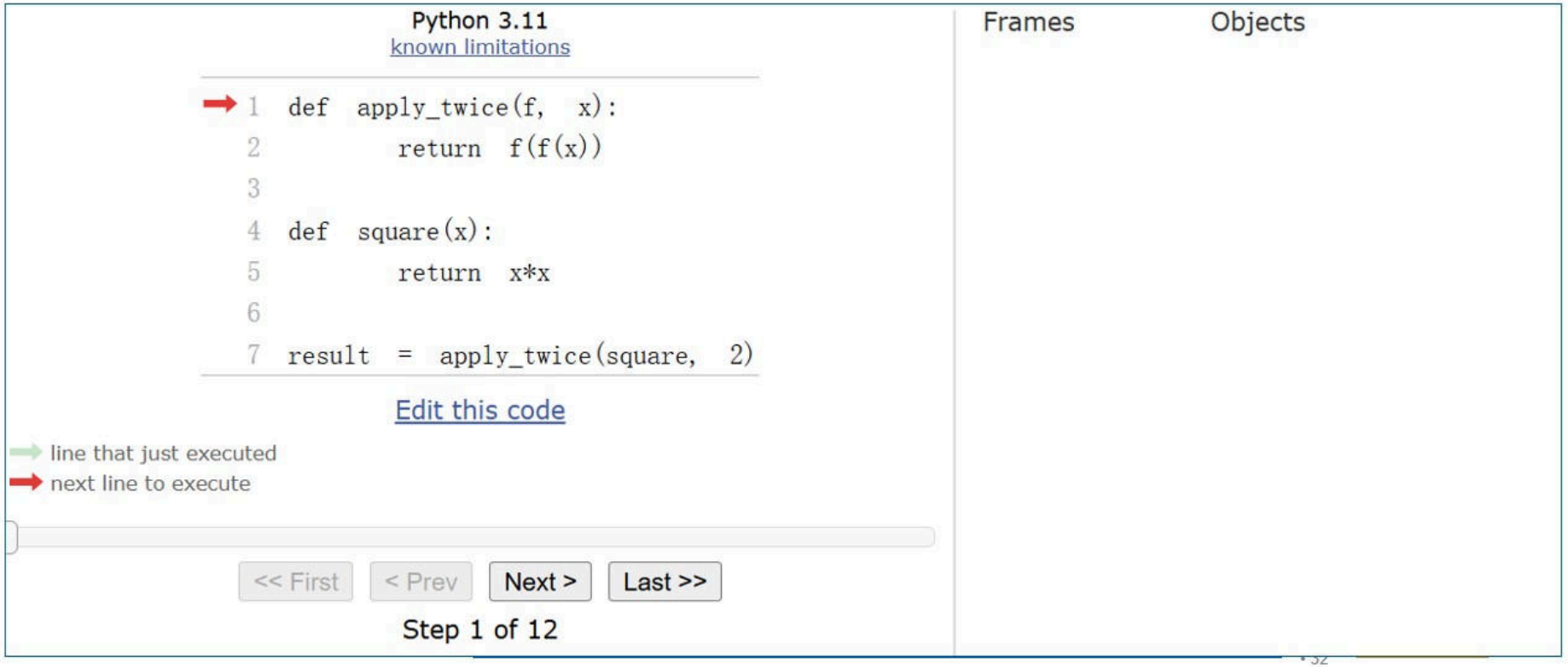


应用用户自定义函数:
- 创建新帧
- 将形式参数（f 和 x）绑定到实际参数
- 执行函数体：return f(f(x))

https://pythontutor.com/

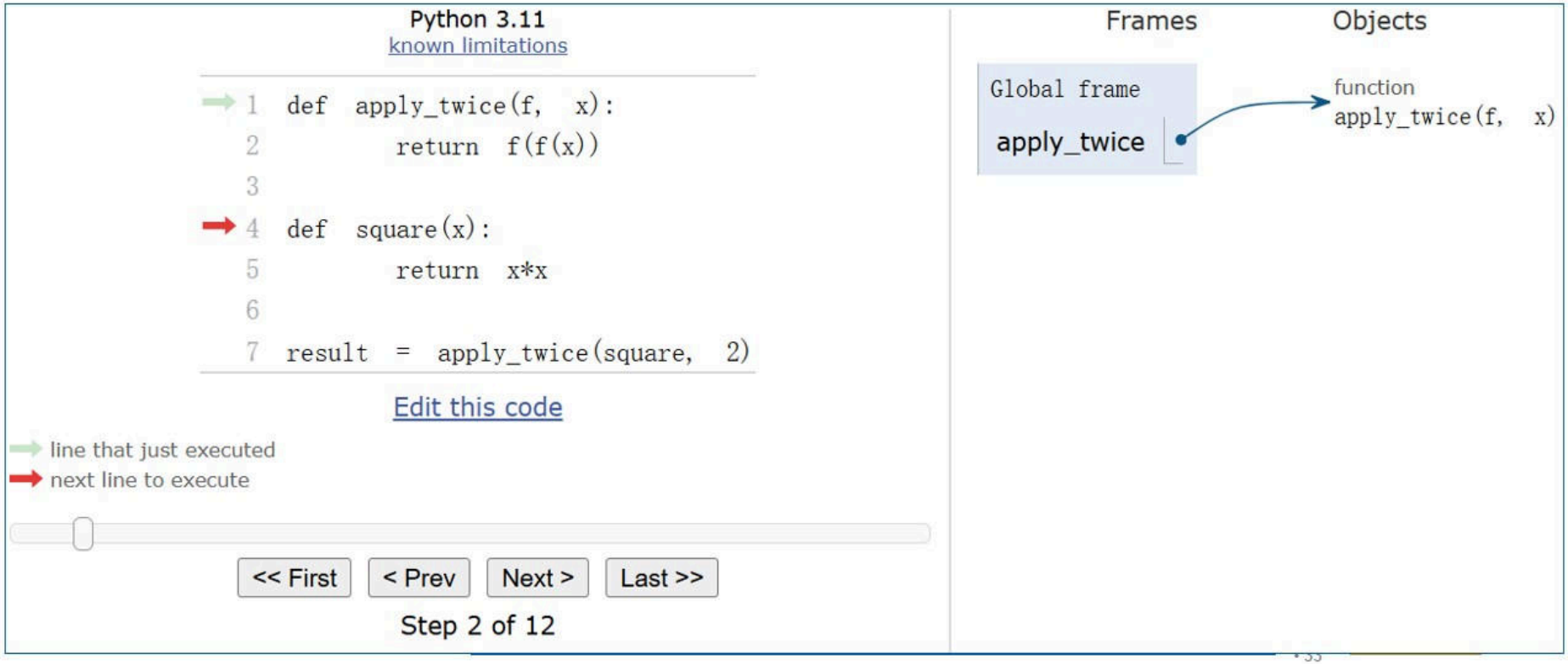
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



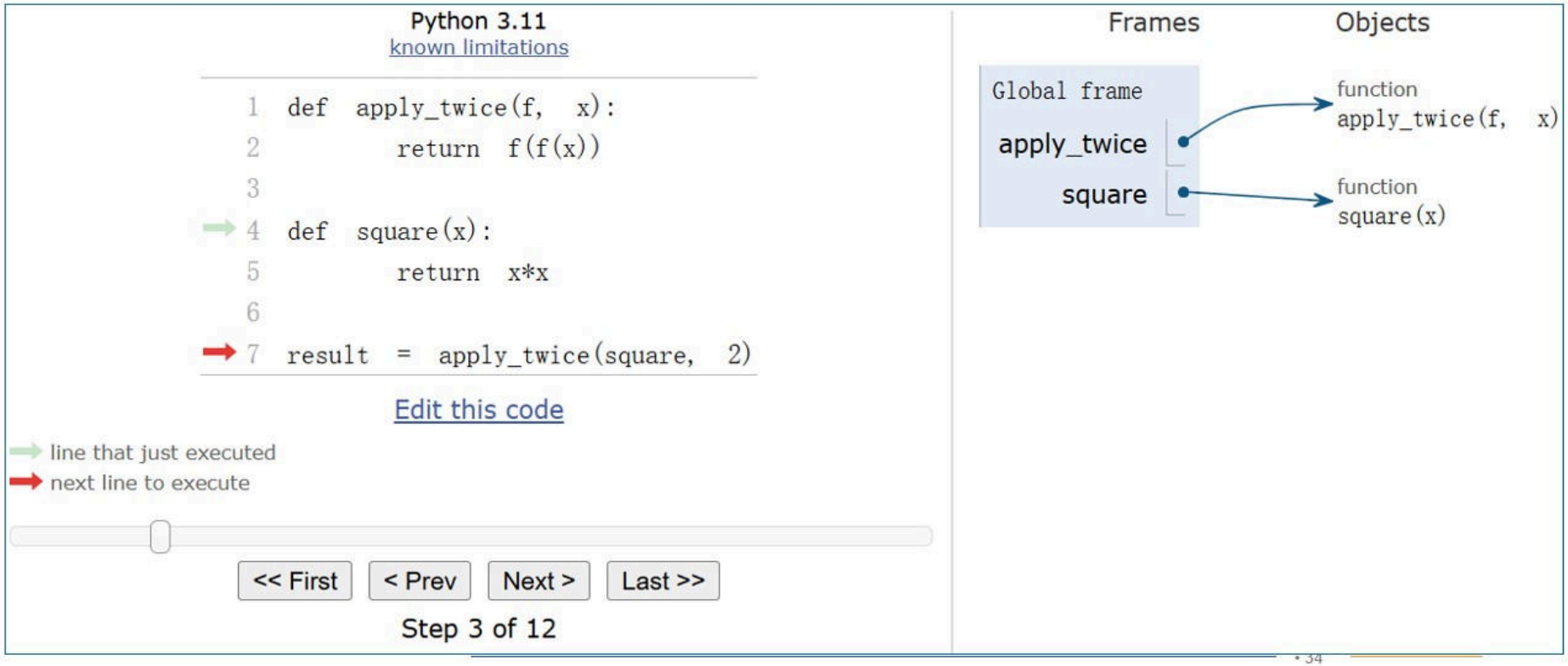
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



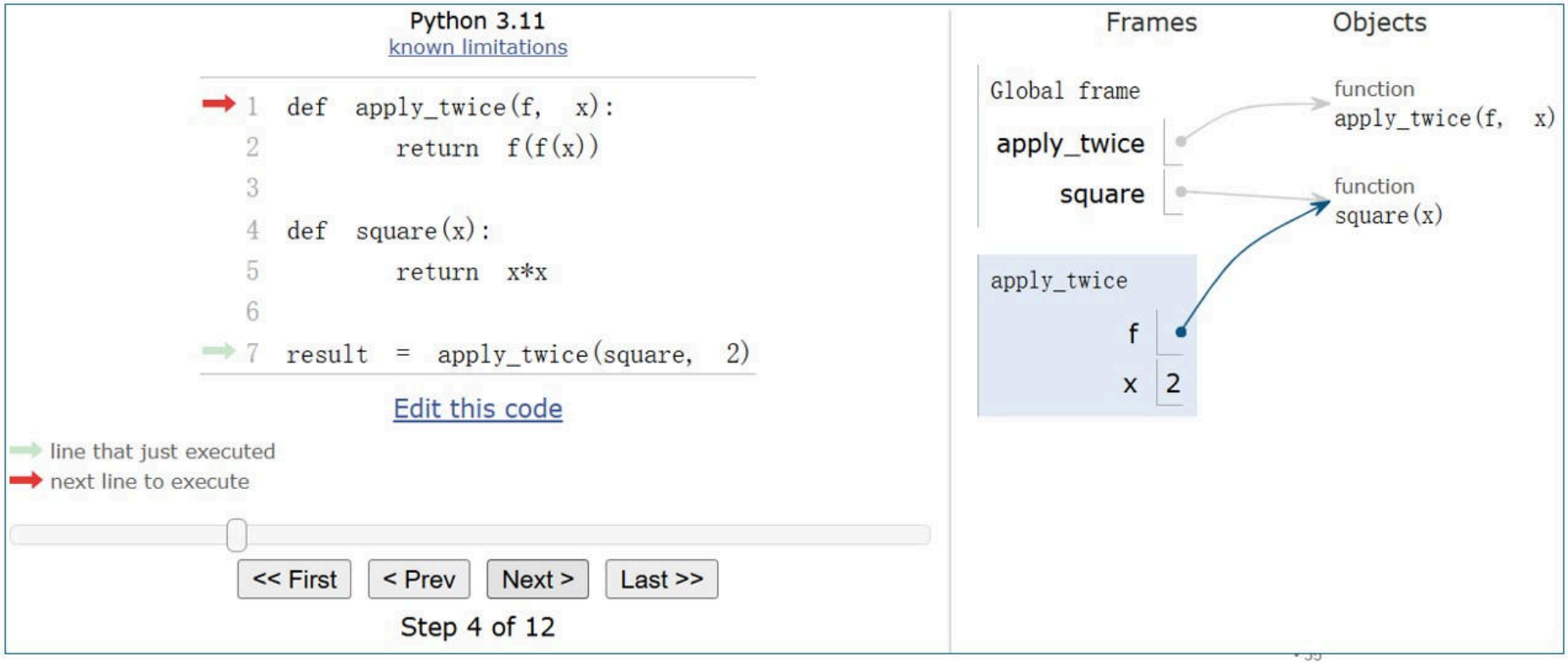
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



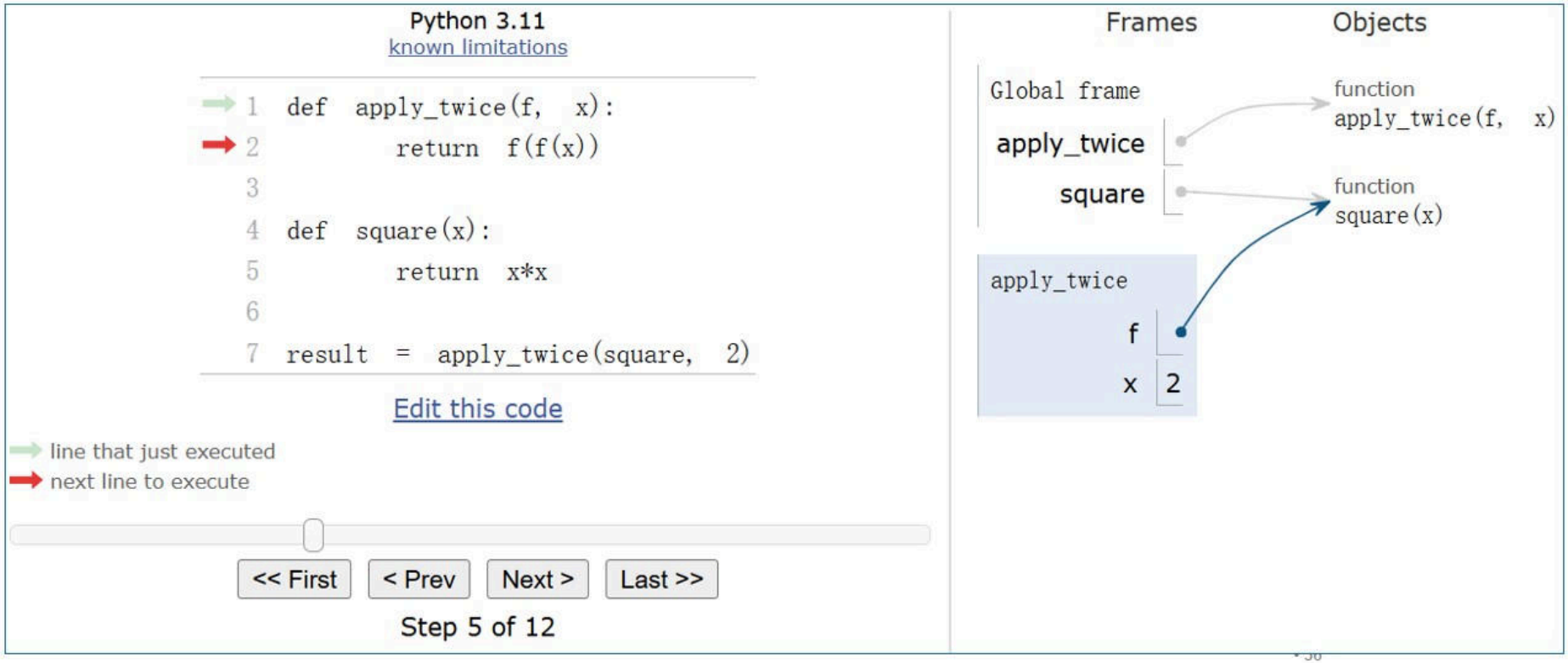
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



Passing Arguments (传递实参)

7. Passing a function as an argument to another function



Passing Arguments (传递实参)

7. Passing a function as an argument to another function



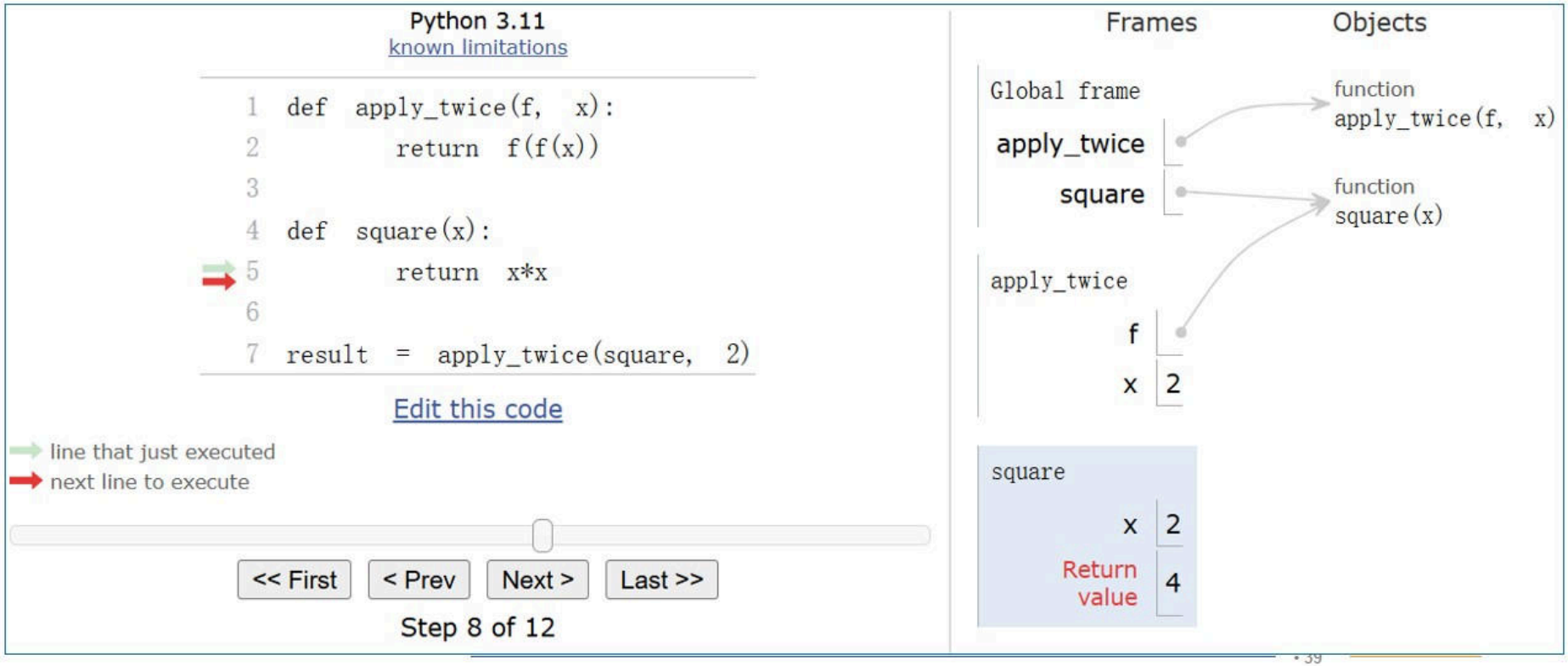
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



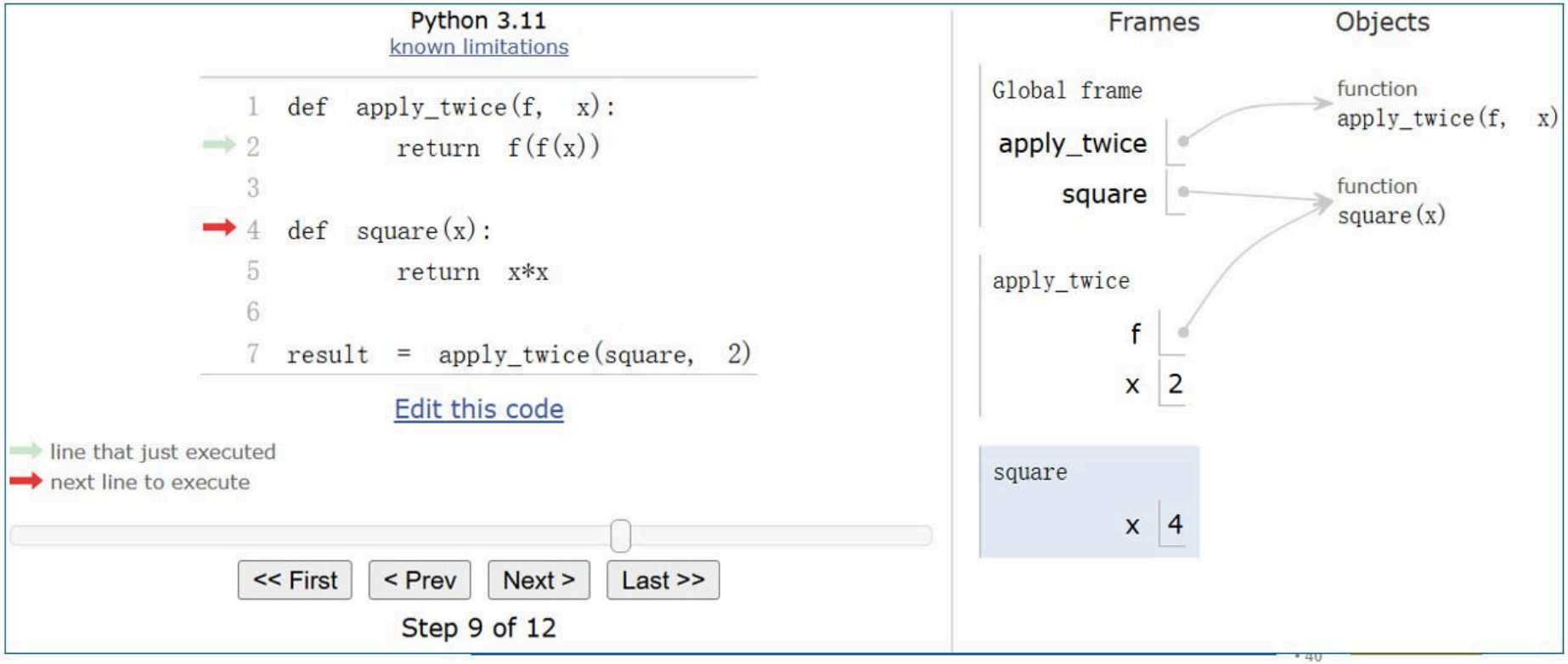
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



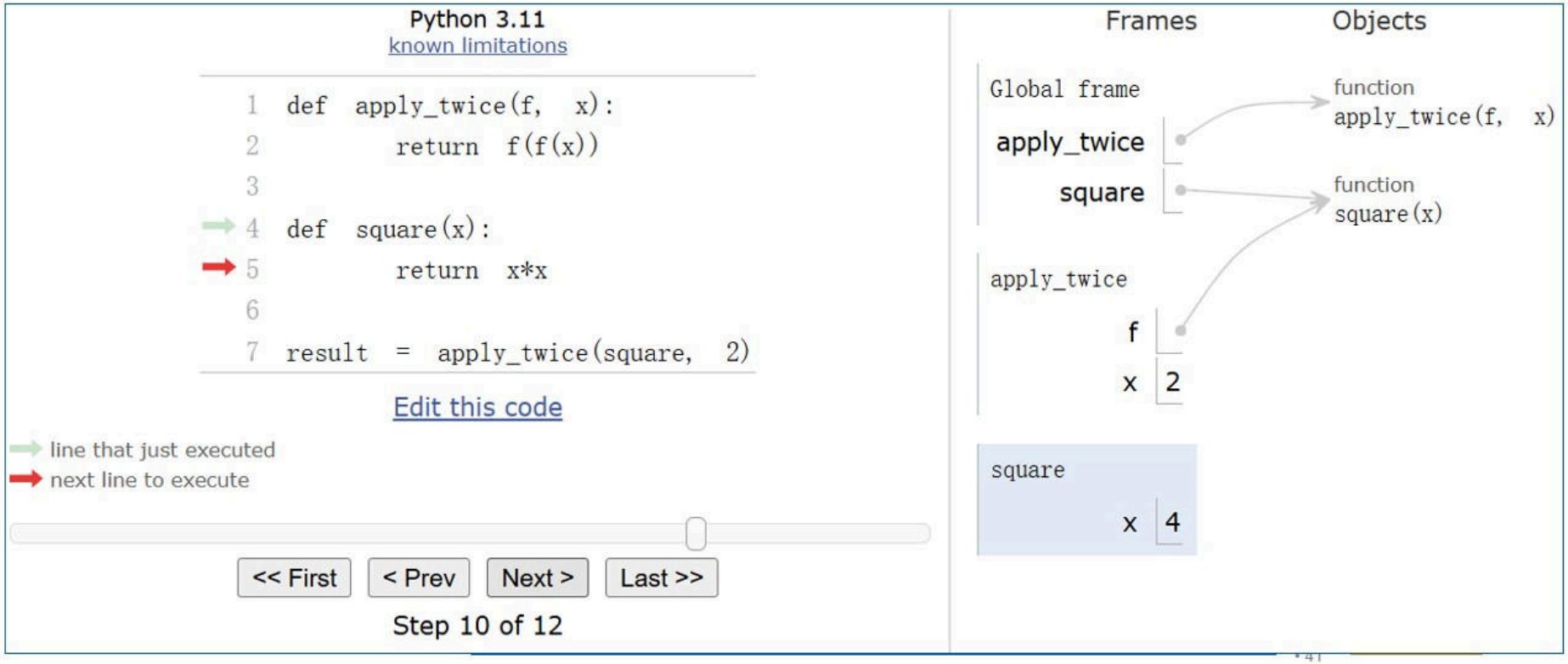
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



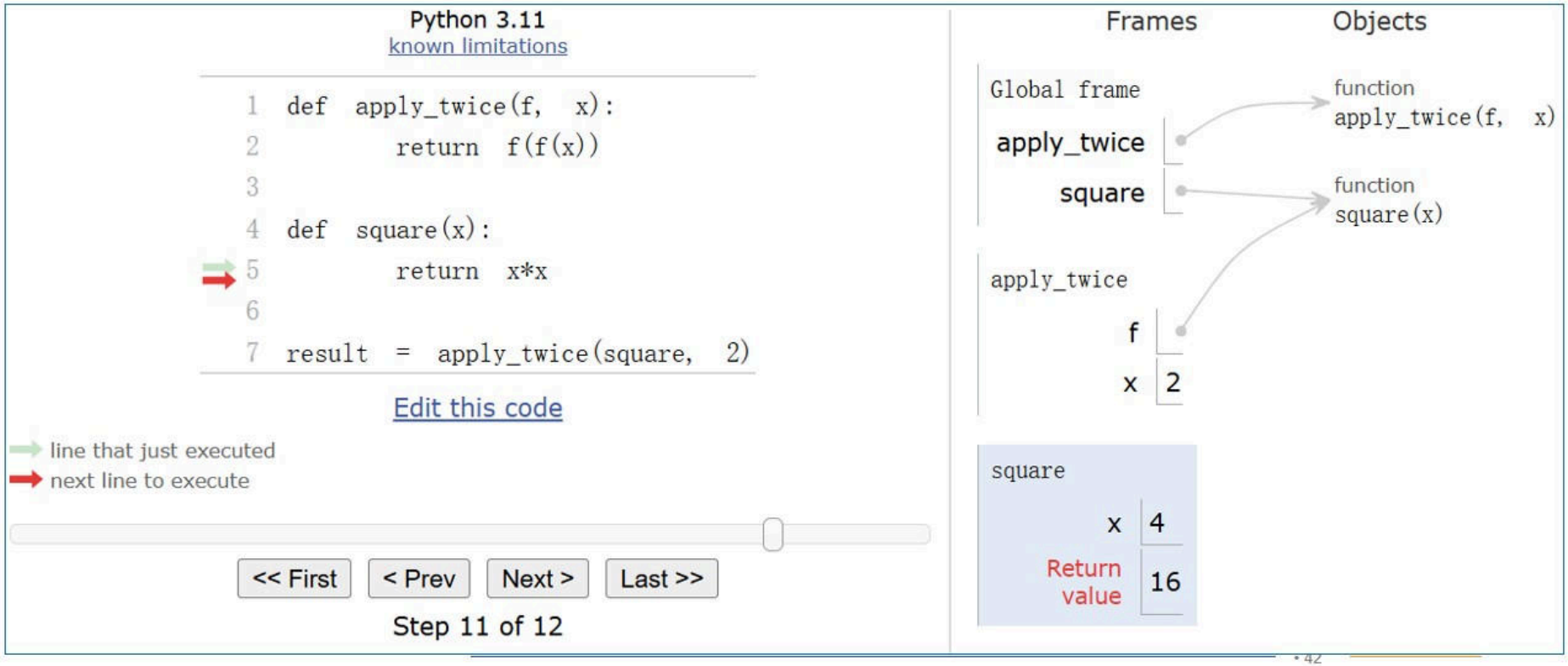
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



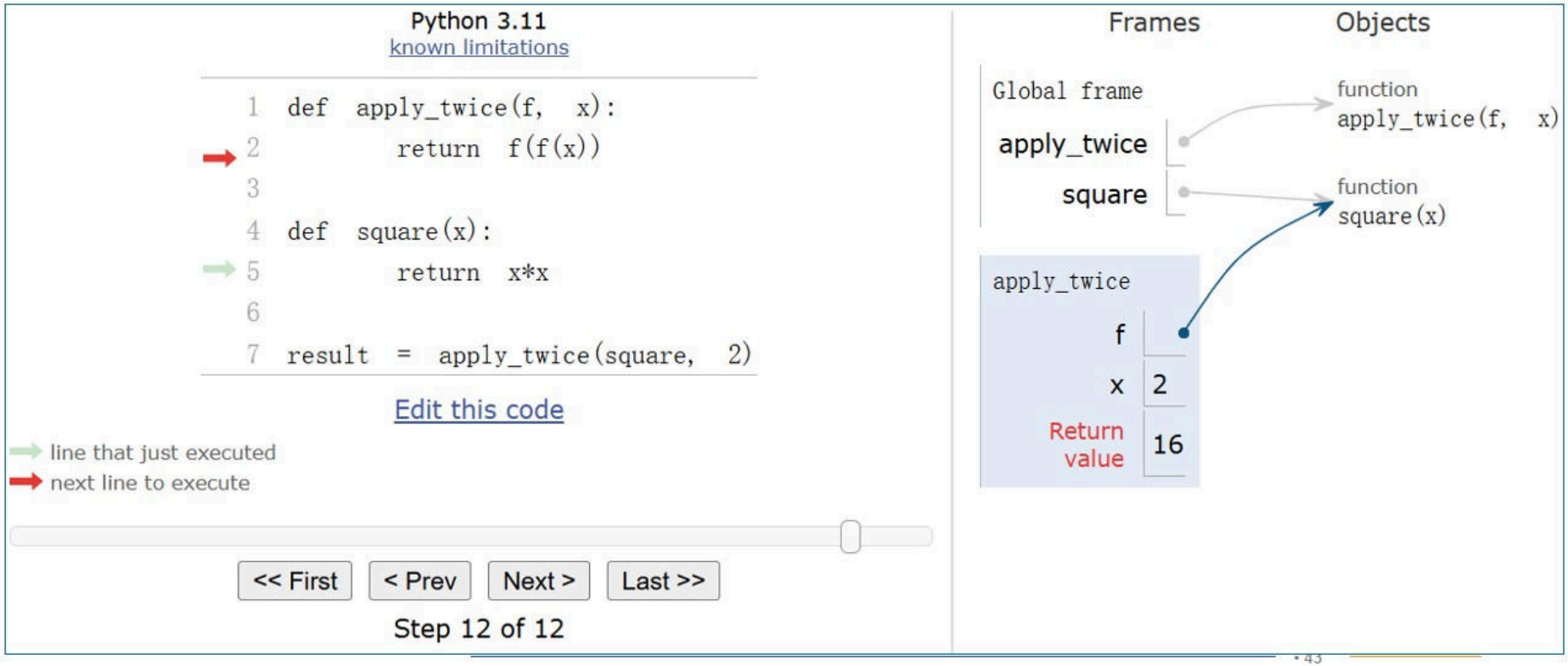
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



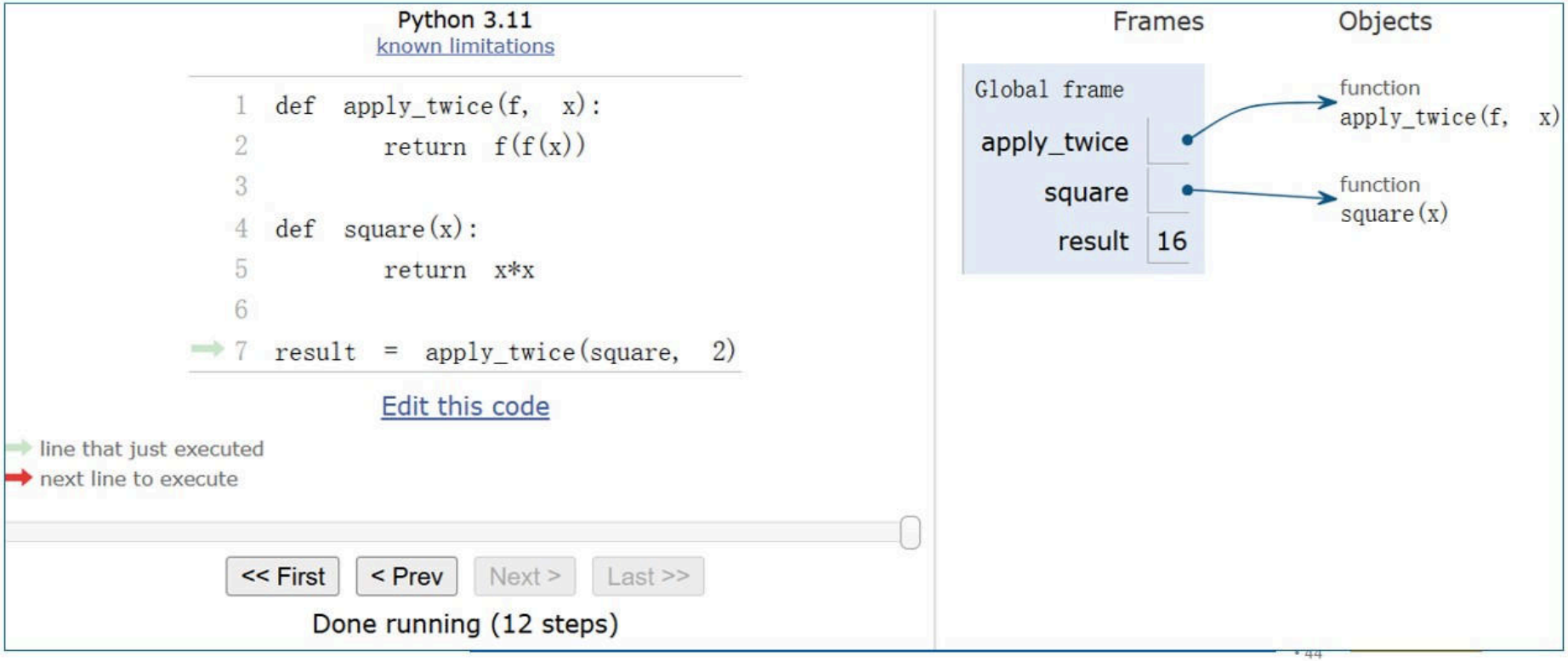
Passing Arguments (传递实参)

7. Passing a function as an argument to another function



Passing Arguments (传递实参)

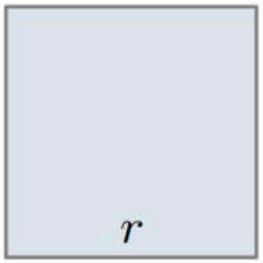
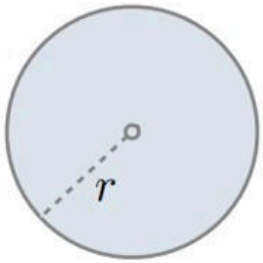
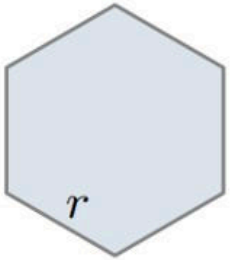
7. Passing a function as an argument to another function



Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments (通过参数来泛化模式)

- Regular geometric shapes relate length and area.

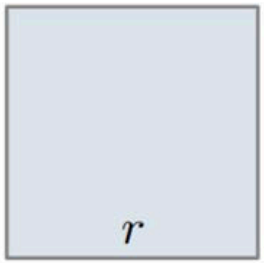
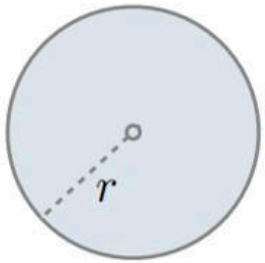
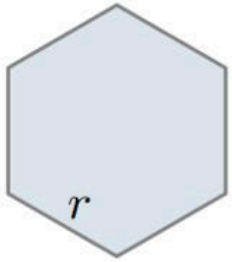
Shape:			
Area:	r^2	$\pi \cdot r^2$	$\frac{3\sqrt{3}}{2} \cdot r^2$

雨课堂 Rain Classroom

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments (通过参数来泛化模式)

- Regular geometric shapes relate length and area.

Shape:			
Area:	$1 \cdot r^2$	$\pi \cdot r^2$	$\frac{3\sqrt{3}}{2} \cdot r^2$

- Finding common structure allows for shared implementation

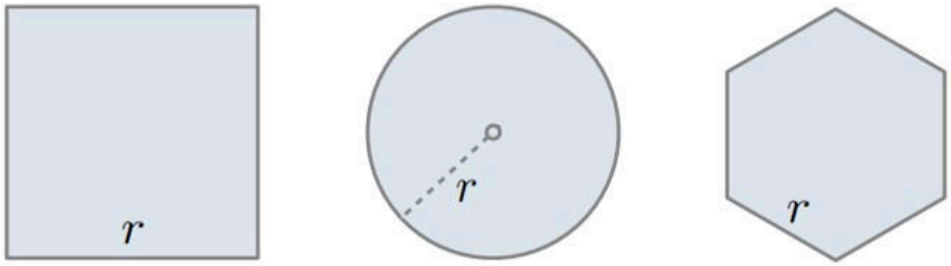
雨课堂 Rain Classroom

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments

- Regular geometric shapes relate length and area.

Shape:




```
def Area(k, r):
    return k*(r**2)
```

Area: $1 \cdot r^2$ $\pi \cdot r^2$ $\frac{3\sqrt{3}}{2} \cdot r^2$

- Finding common structure allows for shared implementation

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments

- The common structure among functions may be a computational process, rather than a number.

$$\sum_{k=1}^{5} k = 1+2+3+4+5 \qquad = 15$$

$$\sum_{k=1}^{5} k^3 = 1^3+2^3+3^3+4^3+5^3 \qquad = 225$$

$$\sum_{k=1}^{5} \frac{8}{(4k-3)\cdot(4k-1)} = \frac{8}{3}+\frac{8}{35}+\frac{8}{99}+\frac{8}{195}+\frac{8}{323} \qquad = 3.04$$

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments

- The common structure among functions may be a computational process, rather than a number.

$$\sum_{k=1}^{5} k = 1+2+3+4+5 = 15$$

$$\sum_{k=1}^{5} k^3 = 1^3+2^3+3^3+4^3+5^3 = 225$$

$$\sum_{k=1}^{5} \frac{8}{(4k-3)\cdot(4k-1)} = \frac{8}{3}+\frac{8}{35}+\frac{8}{99}+\frac{8}{195}+\frac{8}{323} = 3.04$$

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments

$$\sum_{k=1}^{5} k^3 = 1^3 + 2^3 + 3^3 + 4^3 + 5^3 = 225$$

```
def cube(k):
    return pow(k, 3)

def summation(n, term):
    """Sum the first n terms of a sequence.

    >>> summation(5, cube)
    225
    """
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total
```


雨课堂 Rain Classroom

Passing Arguments (传递实参)

8. Generalizing Patterns with Arguments

```
def cube(k):
    return pow(k, 3)

def summation(n, term):
    """Sum the first n terms of a sequence.

    >>> summation(5, cube)
    225
    """
    total, k = 0, 1
    while k <= n:
        total, k = total + term(k), k + 1
    return total
```

Function of a single argument (*not called "term"*)

A formal parameter that will be bound to a function

The cube function is passed as an argument value

The function bound to term gets called here

雨课堂 Rain Classroom

Passing Arguments (传递实参)

9. Passing a function as an argument to another function

```
def hi():
    print ("hi yasoob!")

def doSomethingBeforeHi(func):
    print("I am doing some boring work before executing hi()")
    func()

doSomethingBeforeHi(hi)
```

Decorator prototype(装饰器雏形)

- 装饰器可以在不改动函数的前提下，对函数功能进行扩充
- 例如这个实例，就是在没有改变func函数的前提下，先做了点事情

Output Result:

```
I am doing some boring work before executing hi()
hi yasoob!
```


雨课堂 Rain Classroom

多选题 1分

关于Python中的函数参数和返回值，下列说法正确的是?

A 函数参数数量任意

B 函数参数需要指定类型

C 函数可以没有return语句

D 函数返回值类型需要指定

单选题 1分

在Python中，函数调用必须在函数定义之后进行，否则会出现什么错误?

A SyntaxError

B TypeError

C NameError

D ValueError

单选题 1分

在函数**定义**中，括号内的变量被称为什么?

A Arguments

B Parameters

C Variables

D Constants

填空题 1分

在调用函数时，如果不指定参数名，参数将按照[填空1]的顺序进行匹配。

多选题 1分

在Python中，如何定义一个可以接受任意数量位置参数的函数?

A 使用单个星号（*）作为参数

B 使用双星号（**）作为参数

C 使用逗号分隔多个参数

D 使用方括号[]作为参数

单选题 1分

在Python中，传递参数时，以下哪种说法是正确的?

A 参数传递是通过引用传递的

B 参数传递是通过值传递的

C 参数传递是通过指针传递的

D 参数传递是通过复制传递的

单选题 1分

在函数中传递参数时，以下哪种类型的参数在函数内部被修改后，函数外部的值也会改变?

A 整数

B 字符串

C 列表

D 元组

单选题 1分

在Python中，以下哪个对象是可变的?

A 字符串

B 列表

C 元组

D 整数

雨课堂 Rain Classroom

多选题 1分

关于高阶函数，下列说法正确的是：

A 高阶函数可以返回一个函数作为返回值

B 高阶函数可以接受一个函数作为参数

C 高阶函数只能用于数学运算

D 高阶函数不能返回任何值

雨课堂 Rain Classroom

多选题 1分

关于将函数作为参数传递给另一个函数，下列说法正确的是?

A 可以在不调用函数的情况下传递

B 传递的函数必须有返回值

C 传递的函数可以在另一个函数中被调用

D 传递的函数不能有参数

雨课堂 Rain Classroom

Chapter 4: Functions and Modules

Contents

1. Functions
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. **Python decorators**
   4. Lambda expressions
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. Modules
6. Game example

雨课堂 Rain Classroom

Passing Arguments (传递实参)

9. Passing a function as an argument to another function

```
def hi():
    print ("hi yasoob!")

def doSomethingBeforeHi(func):
    print("I am doing some boring work before executing hi()")
    func()

doSomethingBeforeHi(hi)
```

Decorator prototype(装饰器雏形)

- 装饰器可以在不改动函数的前提下，对函数功能进行扩充
- 例如这个实例，就是在没有改变func函数的前提下，先做了点事情

Output Result:

```
I am doing some boring work before executing hi()
hi yasoob!
```


雨课堂 Rain Classroom

补充：Python decorators (装饰器)

More useful decorators

```
def a_new_decorator(a_func):
    def wrapTheFunction():
        print("I am doing some boring work before executing a_func()")
        a_func()
        print("I am doing some boring work after executing a_func()")
    return wrapTheFunction

def a_func_requiring_decoration():
    print("I am the function which needs some decoration to beautify")

a_func_requiring_decoration = a_new_decorator(a_func_requiring_decoration)

# main
a_func_requiring_decoration()
```

定义一个装饰函数：输入被装饰函数，输出装饰好的函数

定义一个需要装饰的函数

该函数被 wrapTheFunction() 装饰

```
Output:  I am doing some boring work before executing a_func()
         I am the function which needs some decoration to beautify
         I am doing some boring work after executing a_func()
```

补充：Python decorators (装饰器)

More useful decorators

```
def a_new_decorator(a_func):
    def wrapTheFunction():
        print("I am doing some boring work before executing a_func()")
        a_func()
        print("I am doing some boring work after executing a_func()")
    return wrapTheFunction

@a_new_decorator
def a_func_requiring_decoration():
    print("I am the function which needs some decoration to beautify")

# main
a_func_requiring_decoration()
```

a_func_requiring_decoration = a_new_decorator(a_func_requiring_decoration)

Output: I am doing some boring work before executing a_func()
I am the function which needs some decoration to beautify
I am doing some boring work after executing a_func()

补充：Python decorators (装饰器)

More useful decorators

```
def a_new_decorator(a_func):
    def wrapTheFunction():
        print("I am doing some boring work before executing a_func()")
        a_func()
        print("I am doing some boring work after executing a_func()")
    return wrapTheFunction

@a_new_decorator

def a_func_requiring_decoration():
    print("I am the function which needs some decoration to beautify")

# main
print(a_func_requiring_decoration.__name__)
Output:  wrapTheFunction
```

不是我们想要的，我们希望被装饰的函数看起来什么都没变

雨课堂 Rain Classroom

补充：Python decorators (装饰器)

More useful decorators

```
from functools import wraps
def a_new_decorator(a_func):
    @wraps(a_func)
    def wrapTheFunction():
        print("I am doing some boring work before
        a_func()
        print("I am doing some boring work after ex
    return wrapTheFunction

@a_new_decorator

def a_func_requiring_decoration():
    print("I am the function which needs some dec

# main
print(a_func_requiring_decoration.__name__)
Output:  a_func_requiring_decoration
```

Python提供的装饰器

@wraps 接受一个函数来进行装饰，并加入了复制函数名称、注释文档、参数列表等等的功能。这可以让我们在装饰器里面访问在装饰之前的函数的原有属性。

无 @wraps(a_func):

```
print(a_func_requiring_decoration.__name__)
```

Output Result： *wrapTheFunction*

添加@wraps(a_func):

```
print(a_func_requiring_decoration.__name__)
```

Output Result： *a_func_requiring_decoration*

雨课堂 Rain Classroom

补充： Python decorators (装饰器)

Application Scenarios Ⅰ for Decorators —— Authorize(授权)

```
from functools import wraps
def requires_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth = request.authorization
        if not auth or not check_auth(auth.username, auth.password):
            authenticate()    #若授权信息无效，就进行身份验证操作
        return f(*args, **kwargs)
    return decorated
```

补充：Python decorators (装饰器)

Application Scenarios Ⅱ for Decorators —— Log (日志)

```
from functools import wraps
def logit(func):
    @wraps(func)
    def with_logging(*args, **kwargs):
        print(func.__name__ + " was called")
        return func(*args, **kwargs)
    return with_logging

@logit
def addition_func(x):                # Do some math.
  return x + x

result = addition_func(4)
Output Result: addition_func was called
```


雨课堂 Rain Classroom

Define Functions in a Function

```
def hi(name="yasoob"):
    print("now you are inside the hi() function")
    def greet():
        return "now you are in the greet() function"
    def welcome():
        return "now you are in the welcome() function"
    print(greet())
    print(welcome())
    print("now you are back in the hi() function")

# main
hi()
```

无论何时调用hi(), greet()和welcome()将会同时被调用

Output Result:

```
now you are inside the hi() function
now you are in the greet() function
now you are in the welcome() function
now you are back in the hi() function
```

greet() 和 welcome() 函数在 hi() 函数之外是不能访问的

```
# main
greet()
```

Output Result:

```
NameError: name 'greet' is not defined
```


雨课堂 Rain Classroom

多选题 1分

关于Python装饰器的说法，以下哪些是正确的?

A 装饰器可以在函数执行前后添加额外的功能

B 装饰器必须返回一个函数

C 装饰器可以用于类和方法

D 装饰器只能用于函数

雨课堂 Rain Classroom

Chapter 4: Functions and Modules

Contents

1. Functions
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. **Lambda expressions**
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. Modules
6. Game example

雨课堂 Rain Classroom

Lambda expressions (匿名函数)

- **Lambda expressions** (sometimes called lambda forms) are used to create anonymous functions.
- The expression lambda parameters: expression yields a function object.
- The unnamed object behaves like a function object defined with:

  def *<lambda>(parameters):*
      return *expression*

- Example:

```
f = lambda x, y, z: x+y+z
print (f(1, 2, 3))
```

equivalence

```
def f(x, y, z):
    return x+y+z
print (f(1, 2, 3))
```

- 只可以包含一个表达式，表达式的计算结果为函数的返回值；
- 不允许包含其他复杂的语句，但在表达式中可以调用其他函数。

Chapter 4: Functions and Modules

Contents

1. Functions
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. Lambda expressions
   5. **Return Values**
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. Modules
6. Game example

雨课堂 Rain Classroom

Return Values

1. Returning a Simple Value

- A function doesn't always have to display its output directly.
- Instead, it can process some data and then return a value or set of values.
- The value the function returns is called a ***return value***.

```
def get_formatted_name(first_name, last_name):
    """返回整洁的姓名"""
    full_name = first_name + ' ' + last_name
    return full_name.title()

# main
musician = get_formatted_name('jimi', 'hendrix')
print(musician)
```

Output Result:

Jimi Hendrix

Return Values

2. Returning a Dictionary

- A function can return any kind of value you need it to, including more complicated data structures like lists and dictionaries.

```
def build_person(first_name, last_name):
    """Return a dictionary of information about a person."""
    person = {'first': first_name, 'last': last_name}
    return person

# main
musician = build_person('jimi', 'hendrix')
print(musician)
```

Output Result:

```
{'first': 'jimi', 'last': 'hendrix'}
```

Return Values

3. Returning a List

- Functions in Python can return *multiple values* which will be returned in a tuple or other type of collection.

```
def demo(s):    # Count the number of uppercase and lowercase letters in a string
   result = [0, 0]
   for ch in s:
      if 'a' <= ch <= 'z':
         result[0] += 1
      elif 'A' <= ch <= 'Z':
         result[1] += 1
   return result            # Returning a List

# main
print(demo('aaaassssC'))
```

Output Result: [8, 1]

Return Values

4. Returning a function

```
def hi(name="yasoob"):
    def greet():
        return "now you are in the greet() function"

    def welcome():
        return "now you are in the welcome() function"

    if name == "yasoob":
        return greet          # 没加括号——不执行，只是传递
    else:
        return welcome        # 没加括号——不执行，只是传递
# main
func1 = hi()
func1()                       # Output: 'now you are in the greet() function'
func2 = hi("jly")
func2()                       # Output: 'now you are in the welcome() function'
```


雨课堂 Rain Classroom

Return Values

4. Returning a function

- You can use a lambda expression as the return value of a function

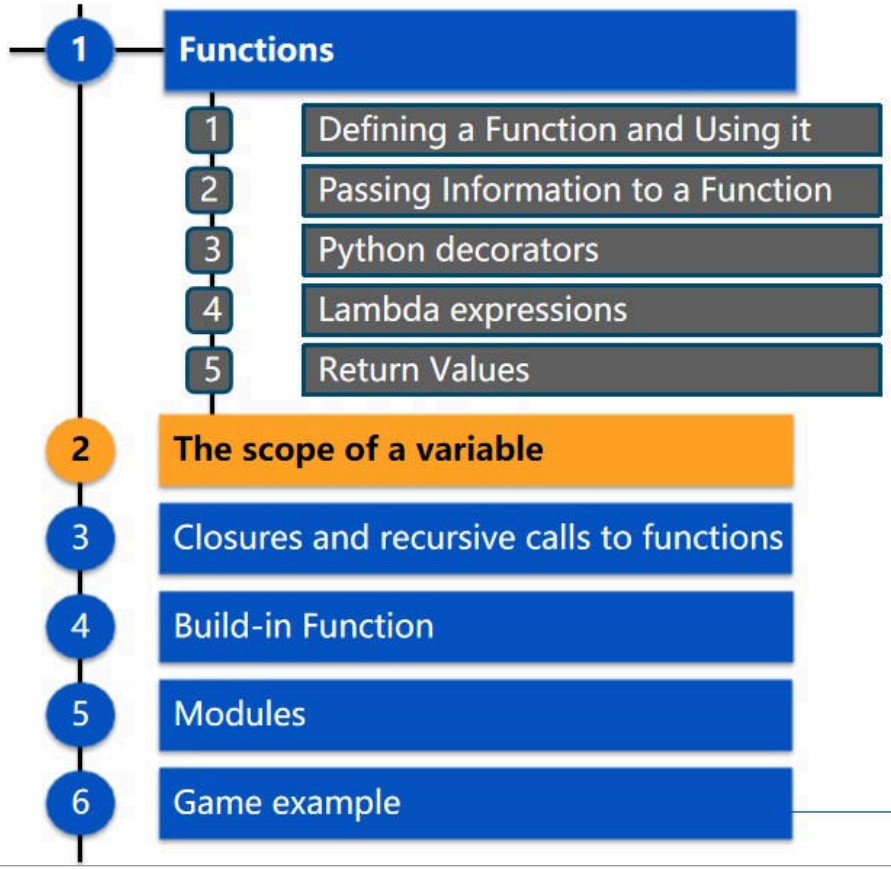
```
def math_lib(k):
    if(k==1): return lambda x, y : x+y
    if(k==2): return lambda x, y : x-y
    if(k==3): return lambda x, y : x*y
    if(k==4): return lambda x, y : x/y
# Call functions
action = math_lib(1)                    # return the addition function
print("10+2=", action(10, 2))
action = math_lib(2)                    # return the subtraction function
print("10-2=",action(10, 2))
action = math_lib(3)                    # return the multiplication function
print("10*2=",action(10, 2))
action = math_lib(4)                    # return the division function
print("10/2=",action(10, 2))
```

相当于返回一个函数

```
Output Result:
10+2= 12
10-2= 8
10*2= 20
10/2= 5.0
```

Chapter 4: Functions and Modules

Contents



雨课堂 Rain Classroom

The scope of a variable

- *The scope of the variable*: the code range in which the variable works.
- The variable name in different scopes can be the same without affecting each other
- Sometimes it is also possible to change the scope of a variable with special keywords.

The scope of a variable

- There are 4 types of scopes of Python variables, they are:
  - L (Local): 函数内的区域，包括局部变量和参数
  - E (Enclosing): 外面嵌套函数的区域，常见的是闭包函数的外层函数
  - G (Glogbal): 全局作用域
  - B (Built-in): 内建作用域

- Python中变量采用L-->E-->G-->B的规则查找，即Python检索变量的时候，会优先在局部作用域查找，如果没找到，便会去局部外的区域查找（例如闭包），再找不到就会去全局作用域中找，最后再去内建作用域中找。

The scope of a variable

1. Local variables

- Local variables: ordinary variables defined inside the function, only work inside the function
- References to local variables are faster than global variables

```
def fun():
    x = 3          # 函数内定义的变量作用域只在函数内，函数结束，自动删除
    count = 2
    while count > 0:
        print (x)
        count = count - 1

# main
fun()
print (x)
```

```
Output Result:
3
3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'x' is not defined
```

The scope of a variable

2. Global variables

- A variable has been defined outside the function, if you need to assign a value to this variable in the function, and reflect the result of the assignment outside the function, can be defined by the *keyword* ***global***.

```
x = 2              # Global variables
def fun1():
   print (x)       # Access global variable x
def fun2():
   x = x+1         # Variables assigned within a function are considered local variables
   print (x)
# main
fun1()             # Output: 2
fun2()             # Output:
print (x)          # Output: 2
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in fun2
UnboundLocalError: local variable 'x' referenced before assignment
```

The scope of a variable

2. Global variables

```
x = 2                    # Global variables
def fun1():
    print (x, end=" ")
def fun2():
    x = 2
    x = x+1
    print (x, end=" ")
# main
fun1()                   # Output: 2
fun2()                   # Output: 3
print (x, end=" ")       # Output: 2
```

- 在Python中，如果在函数内部对全局变量进行修改，Python会把此变量当做局部变量。

雨课堂 Rain Classroom

The scope of a variable

2. Global variables

```
x = 2                    # Global variables
def fun1():
    print (x, end=" ")
def fun2():
    global x
    x = x+1
    print (x, end=" ")
# main
fun1()                   # Output: 2
fun2()                   # Output: 3
print (x, end=" ")       # Output: 3
```

- 为了使全局变量生效，可以在函数内使用global关键字进行声明。

雨课堂 Rain Classroom

The scope of a variable

2. Global variables

- Variables specified with the *global keyword* may not exist before

```
def fun1():
    global x
    x = 2
    print (x, end=" ")
def fun2():
    global x
    x = x+1
    print (x, end=" ")
fun1()
fun2()
print (x, end=" ")

Output Result:
2 3 3
```

```
def fun1():
    global x
    x=2
    print (x, end=" ")
def fun2():
    x = 4
    x=x+1
    print (x, end=" ")
fun1()
fun2()
print (x, end=" ")

Output Result:
2 5 2
```


雨课堂 Rain Classroom

The scope of a variable

3. Keyword *nonlocal*

- *nonlocal*: keyword that modifies the scope of a variable in a nested function.

```
def fun1():
    count = 1
    def fun_in():
        count = 12
    fun_in()
    print (count)

>>>fun1()

Output Result: 1
```

```
def fun1():
    count = 1
    def fun_in():
        count = 12
        def fun_inn():
            nonlocal count
            count = 14
    fun_in()
    print (count)

>>>fun1()

Output Result: 12
```

Chapter 4: Functions and Modules

Contents

1 Functions
- 1 Defining a Function and Using it
- 2 Passing Information to a Function
- 3 Python decorators
- 4 Lambda expressions
- 5 Return Values

2 The scope of a variable

3 **Closures and recursive calls to functions**

4 Build-in Function

5 Modules

6 Game example

雨课堂 Rain Classroom

Closures and recursive calls to functions

1. Closure（闭包）

```
def func_out(n=0):
    count = [n]      # 外部函数的变量
    def func_inner( ):          # ①
        count[0] += 1           # ②
        print(count[0])
    return func_inner           # ③
# main
quote = func_out(5)
quote()
quote()
Output Result:
6
7
```

- 在Python中，函数支持嵌套。
- 如果在一个内部函数中对外部函数作用域（非全局作用域）的变量进行引用，那么内部函数**称为闭包。闭包满足三个条件：**

① 存在于嵌套关系的函数中（内部函数）

② 嵌套的内部函数引用了外部函数的变量

③ 嵌套的外部函数将内部函数名作为返回值返回

雨课堂 Rain Classroom

Closures and recursive calls to functions

2. Application of closures

```
def test(a, b, c):
    print(a*b+c)
# main
test(1, 2, 1)        # Output: 3
test(1, 2, 2)        # Output: 4
test(1, 2, 3)        # Output: 5
test(1, 2, 4)        # Output: 6
test(2, 3, 2)        # Output: 8
test(2, 3, 3)        # Output: 9
test(2, 3, 4)        # Output: 10
test(2, 3, 5)        # Output: 11
```

```
def test(a, b):
    def test_in(c):
        print(a*b+c)
    return test_in
# main
num = test(1,2)
num(1)          # Output: 3
num(2)          # Output: 4
num(3)          # Output: 5
num(4)          # Output: 6
```

- 把test_in引用交给了num，同时把a, b变量也绑定给了num

- 我们发现一个问题， 前4次函数调用a,b 是固定的，只是c 会变动，后4次函数调用也是如此，如何减少a, b 参数的传递?

雨课堂 Rain Classroom

Closures and recursive calls to functions

2. Application of closures

```
def test(a, b, c):
    print(a*b+c)
# main
test(1, 2, 1)        # Output: 3
test(1, 2, 2)        # Output: 4
test(1, 2, 3)        # Output: 5
test(1, 2, 4)        # Output: 6
test(2, 3, 2)        # Output: 8
test(2, 3, 3)        # Output: 9
test(2, 3, 4)        # Output: 10
test(2, 3, 5)        # Output: 11
```

```
def test(a, b):
    def test_in(c):
        print(a*b+c)
    return test_in
# main
num = test(1,2)
num(1)
num(2)
num(3)
num(4)
```

- 把test_in引用交给了num，同时把a, b变量也绑定给了num

```
# 如果不想用上面的a,b值，可以重新创建引用num2
num2 = test(2,3)
num2(2)        # Output: 8
num2(3)        # Output: 9
num2(4)        # Output: 10
num2(5)        # Output: 11
```

- 我们发现一个问题， 前4次函数调用a,b 是固定的，只是c 会变动，后4次函数调用也是如此，如何减少a, b 参数的传递?

雨课堂 Rain Classroom

Closures and recursive calls to functions

2. Application of closures

```
def test(a, b, c):
    print(a*b+c)
# main
test(1, 2, 1)        # Output: 3
test(1, 2, 2)        # Output: 4
test(1, 2, 3)        # Output: 5
test(1, 2, 4)        # Output: 6
test(2, 3, 2)        # Output: 8
test(2, 3, 3)        # Output: 9
test(2, 3, 4)        # Output: 10
test(2, 3, 5)        # Output: 11
```

```
def test(a, b):
    def test_in(c):
        print(a*b+c)
    return test_in
# main
num = test(1,2)
num(1)          # Output: 3
num(2)
num(3)
num(4)
```

■ 把test_in引用交给了num，同时把a, b变也绑定给了num

```
# 如果不想用上面的a,b值，可以重新创建引用num2
num2 = test(2,3)
num2(2)          # Output: 8
num2(3)
num2(4)
num2(5)
```

```
.....
num2(4)          # Output: 10
num2(5)          # Output: 11
# 甚至可以继续执行第一个num
num(5)           # Output: 7
```

■ 我们发现一个问题， 前4次函数调用a,b 是固定的，只是c 会变动，后4次函数调用也是如此，如何减少a, b 参数的传递?

雨课堂 Rain Classroom

Closures and recursive calls to functions

Function Composition

```
def square(x):
    return x * x

def make_adder(n):
    def adder(k):
        return k + n
    return adder

def compose1(f, g):
    def h(x):
        return f(g(x))
    return h

compose1(square, make_adder(2))(3)
```


雨课堂 Rain Classroom

Closures and recursive calls to functions

3. Recursive calls to functions

- **Recursive calls:**
  - A function calls itself directly or indirectly during its execution
- **Example-1:**
  - Enter an integer from the keyboard and find the factorial(阶乘) of the number
- **Analysis:**
  - The factorial of a number *n* defines ——n! = n (n-1)!

    fac(n) = 1 (n=1)

    fac(n) = n*fac(n-1) (n>1)

Closures and recursive calls to functions

3. Recursive calls to functions

- The factorial of a number *n* defines ——n! = n (n-1)!

  fac(n) = 1 (n=1)

  fac(n) = n*fac(n-1) (n>1)

```
def fac(n):
  if n == 1:                  # Condition for the end of the recursive call(结束条件)
    return 1
  else:
    return(n*fac(n-1))        # Calling the function fac( ) itself

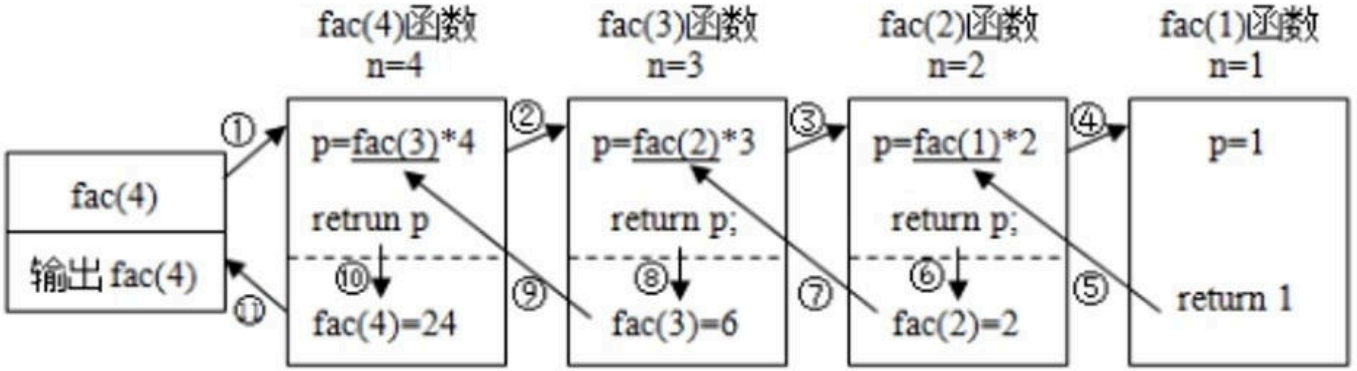
# main
x = int(input("enter a positive integer: "))
print(fac(x))
```


雨课堂 Rain Classroom

Closures and recursive calls to functions

3. Recursive calls to functions

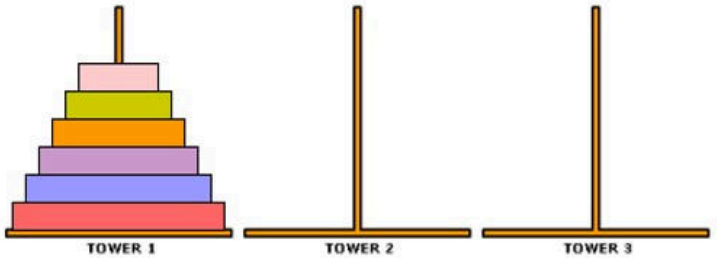
- The execution process of recursive call is divided into two parts:
  - recursion(递推) process and regression(回归) process.
- These two processes are controlled by the recursive abort condition(终止条件).



补充：递归调用实例2——汉诺塔问题

- 将问题简化描述为：
  - n个盘子和3根柱子：A(源)、B(备用)、C(目的)，盘子大小不同，每个盘子只能放在比它大的盘子上面。初始状态，所有盘子在A柱上。
  - **问题是**将盘子一个一个地从A柱子移动到C柱子。移动过程中，可以使用B柱，但盘子也只能放在比它大的盘子上面。

- 因此我们得出汉诺塔问题的以下几个限制条
  1. 在小圆盘上不能放大圆盘。
  2. 在三根柱子之间一回只能移动一个圆盘。
  3. 只能移动在最顶端的圆盘。



补充：递归调用实例2——汉诺塔问题

汉诺塔问题分析：

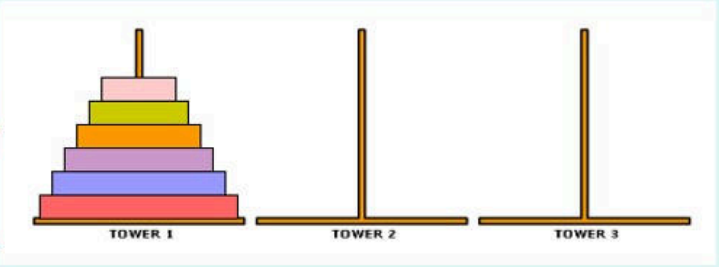
❶A柱只有一个盘子的情况：A柱-->C柱

❷A柱有两个盘子的情况：小盘A柱-->B柱，大盘A柱-->C柱，小盘B柱-->C柱

❸A柱有n个盘子的情况：将此问题看成上面n-1个盘子和最下面第n个盘子的情况。将n-1个盘子A柱-->B柱，第n个盘子A柱-->C柱，n-1个盘子B柱-->C柱

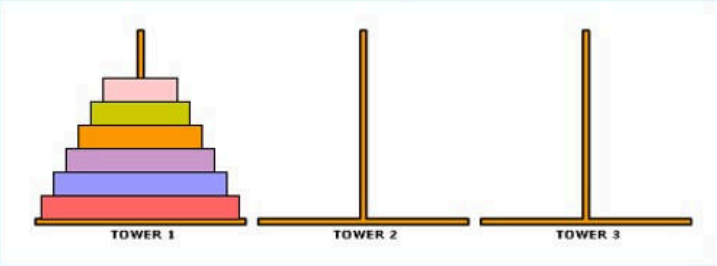
问题转化为搬动n-1个盘子的问题，同样像上面一样处理，直到成为最后搬动一个盘子的情况。

- 这是一个典型的递归问题，递归结束于只搬动



补充：递归调用实例2——汉诺塔问题

汉诺塔问题算法描述：



❶ n-1个盘子A柱-->B柱，借助C柱；

❷ 第n个盘子A柱-->C柱；

❸ n-1个盘子B柱-->C柱，借助A柱。

- 其中步骤❶和❸继续递归下去，直至搬动一个盘子为止。
- 由此，定义两个函数：
  - 一个是递归函数 *hanoi(n,source,temp,target)*，实现将n个盘子从源柱source借助中间柱temp搬到目标柱target；
  - 另一个函数 *move(source,target)*，用来输出搬动一个盘子的提示信息。

补充：递归调用实例2——汉诺塔问题

```
def move(source, target):
    print(source, "-->", target)

def hanoi(n, source, temp, target):
    if(n == 1):
        move(source, target)
    else:
        hanoi(n-1, source, target, temp)        # 将 n-1 个盘子搬到中间柱
        move(source, target)                    # 将最后一个盘子搬到目标柱
        hanoi(n-1, temp, source, target)        # 将 n-1 个盘子搬到目标柱
# main
n = int(input("输入盘子数："))
print("移动", n, "个盘子的步骤是：")
hanoi(n, 'A', 'B', 'C')
```


雨课堂 Rain Classroom

Self-Reference

Returning a Function Using Its Own Name

```
def print_sums(n):
    print(n)
    def next_sum(k):
        return print_sums(n+k)
    return next_sum

print_sums(1)(3)(5)
print_sums(1)(3)(5)(7)
```


雨课堂 Rain Classroom

Chapter 4: Functions and Modules

Contents

1. **Functions**
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. Lambda expressions
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. **Build-in Function**
5. Modules
6. Game example

雨课堂 Rain Classroom

Built-in Functions

- Built-in functions(内置函数), also known as system functions;
- The functions provided by Python itself can be used at any time.
- Python's commonly used built-in functions include mathematical operation functions, conversion functions, and random number functions.
- Log in to the official Python website to view the help documentation：

  https://docs.python.org/3/library/functions.html

雨课堂 Rain Classroom

The Python Standard Library

While The Python Language Reference describes the exact syntax and semantics of the Python language, this library reference manual describes the standard library that is distributed with Python. It also describes some of the optional components that are commonly included in Python distributions.

Python's standard library is very extensive, offering a wide range of facilities as indicated by the long table of contents listed below. The library contains built-in modules (written in C) that provide access to system functionality such as file I/O that would otherwise be inaccessible to Python programmers, as well as modules written in Python that provide standardized solutions for many problems that occur in everyday programming. Some of these modules are explicitly designed to encourage and enhance the portability of Python programs by abstracting away platform-specifics into platform-neutral APIs.

The Python installers for the Windows platform usually include the entire standard library and often also include many additional components. For Unix-like operating systems Python is normally provided as a collection of packages, so it may be necessary to use the packaging tools provided with the operating system to obtain some or all of the optional components.

In addition to the standard library, there is a growing collection of several thousand components (from individual programs and modules to packages and entire application development frameworks), available from the Python Package Index.

- Introduction
  - Notes on availability
- Built-in Functions
- Built-in Constants

Built-in Functions

The Python interpreter has a number of functions and types built into it that are always available. They are listed here in alphabetical order.

Built-in Functions				
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

雨课堂 Rain Classroom

Discussion Question 1

What is the value of the final expression in this sequence?

```
>>> f = min
>>> f = max
>>> g, h = min, max
>>> max = g
>>> max(f(2, g(h(1, 5), 3)), 4)
```

???

Execution rule for assignment statements:

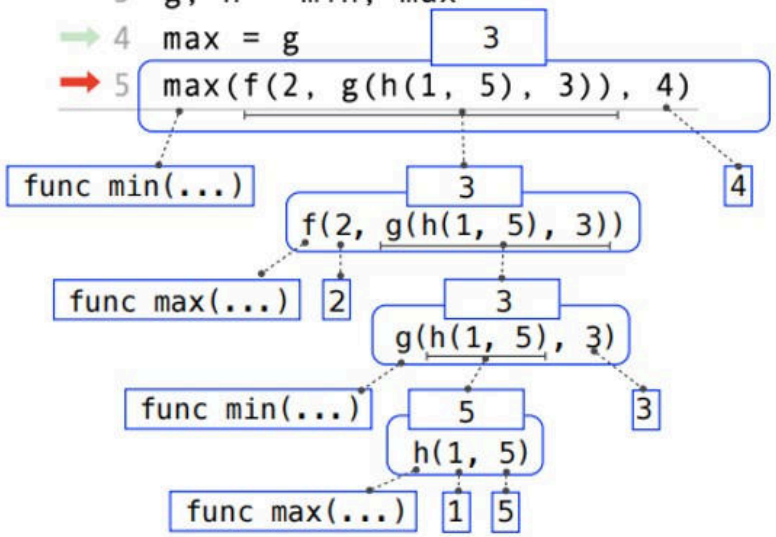
1. Evaluate all expressions to the right of = from left to right.
2. Bind all names to the left of = to those resulting values in the current frame.

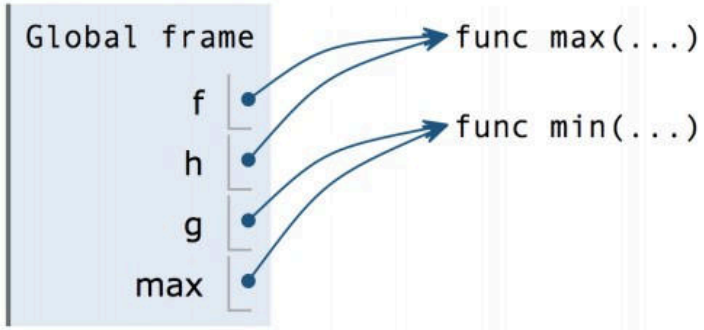
雨课堂 Rain Classroom

Discussion Question 1

What is the value of the final expression in this sequence?

```
1  f = min
2  f = max
3  g, h = min, max
4  max = g
5  max(f(2, g(h(1, 5), 3)), 4)
```





Chapter 4: Functions and Modules

Contents

1. **Functions**
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. Lambda expressions
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. **Modules**
6. Game example

雨课堂 Rain Classroom

Modules

- Modules logically organize your Python code snippets.
- Putting related code in a module makes the code easier to use and easier to understand.
- A module **is a file** that holds Python code.
- Functions, classes and variables can be defined in modules.
- Modules in Python are similar to header files in C language and packages in Java. For example, to call the sqrt function in Python, the math module must be imported with the keyword *import*:

```
import math
print ("50的平方根: ", math.sqrt(50))
y = math.hex(50)
print ("50的16进制: ", y)
```


雨课堂 Rain Classroom

Modules

1. Import modules

- **import module method Ⅰ (Importing an Entire Module)**

  import module_name

  - This way to import a module, when using the functions(class) in the module, you must use:

    module_name.function_name

```
import math
print ("50的平方根: ", math.sqrt(50))
y = math.hex(50)
print ("50的16进制: ", y)
```


雨课堂 Rain Classroom

Modules

1. Import modules

- **import module method Ⅱ (Using as to Give a Module an Alias)**

  import module_name as alias

  - you can use a short, unique alias—an alternate name similar to a nickname for the module:

    alias.function_name

```
import math as m
print ("50的平方根: ", m.sqrt(50))
y = m.hex(50)
print ("50的16进制: ", y)
```


雨课堂 Rain Classroom

Modules

1. Import modules

- **import module method Ⅲ (Importing Specific Functions)**

  from module_name import function_0, function_1, function_2 ....

  - This way to import the functions, when using the functions in the module, you don't need to give the *module_name*

```
from math import degrees, radians, pi
print ("π的角度是", degrees(pi))   # Output: π的角度是 180.0
y = radians(180)
print ("180° = ", y)               # Output: 180° =  3.141592653589793
```


雨课堂 Rain Classroom

Modules

1. Import modules

- **import module method Ⅳ (Using as to Give a Function an Alias)**

  from *module_name* import function_name as alias

  - you can use a short, unique alias—an alternate name similar to a nickname for the function.

```
from math import degrees as deg
print ("π的角度是", deg(pi))
# Output: π的角度是 180.0
```


雨课堂 Rain Classroom

Modules

1. Import modules

- **import module method V (Importing All Functions in a Module)**

  from *module_name* import *

  - The asterisk(星号) in the import statement tells Python to copy every function from the module *math* into the program file.

```
from math import *
print ("π的角度是", degrees(pi))
# Output: π的角度是 180.0
```

- 使用并非自己编写的大型模块时，最好不要采用这种导入方法。
- 因为，如果模块中有函数的名称与你的项目中使用的名称相同，则会覆盖你的函数，产生意想不到的结果，而不是分别导入所有的函数。

  - Because every function is imported, you can call each function by name without using the dot notation.

Modules

2. Search order for module locations

- When importing a module, the Python interpreter searches for module locations in the following order:

❶ Current directory;

❷ Each directory under the *PYTHON PATH* environment variable;

❸ The default directory determined by the installation process

```
import sys
print(sys.path)
```

■ 模块搜索路径存储在system模块的sys.path变量中，变量包含❶❷❸所说的路径

```
In [17]: print(sys.path)
['', 'C:\\Users\\jerry\\Anaconda3\\python36.zip', 'C:\\Users\\jerry\\Anaconda3\\DLLs', 'C:\\Users\\jerry\
\Anaconda3\\lib', 'C:\\Users\\jerry\\Anaconda3', 'C:\\Users\\jerry\\Anaconda3\\lib\\site-packages', 'C:\\
\jerry\\Anaconda3\\lib\\site-packages\\win32', 'C:\\Users\\jerry\\Anaconda3\\lib\\site-packages\\win32\\l
\Users\\jerry\\Anaconda3\\lib\\site-packages\\Pythonwin', 'C:\\Users\\jerry\\Anaconda3\\lib\\site-package
\\extensions', 'C:\\Users\\jerry\\.ipython']
```

Modules

3. List module contents

>>> dir(*module_name*)　　# 函数返回一个排好序的字符串列表，内容是模块里定义的变量和函数

```
import sys
print(dir(sys))
import math
print(dir(math))
print(dir())
```

```
>>> import sys
>>> dir(sys)
['__breakpointhook__', '__displayhook__', '__doc__', '__except
in__', '__stdout__', '__unraisablehook__', '_base_executable',
 '_xoptions', 'abiflags', 'addaudithook', 'api_version', 'argv
ll_tracing', 'copyright', 'displayhook', 'dont_write_bytecode'
', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_depth'
emencoding', 'getprofile', 'getrecursionlimit', 'getrefcount',
intern', 'is_finalizing', 'last_traceback', 'last_type', 'last
latform', 'platlibdir', 'prefix', 'ps1', 'ps2', 'pycache_prefi
rsionlimit', 'setswitchinterval', 'settrace', 'stderr', 'stdin
>>> import math
>>> dir(math)
['__doc__', '__file__', '__loader__', '__name__', '__package__
os', 'cosh', 'degrees', 'dist', 'e', 'erf', 'erfc', 'exp', 'ex
isfinite', 'isinf', 'isnan', 'isqrt', 'lcm', 'ldexp', 'lgamma'
emainder', 'sin', 'sinh', 'sqrt', 'tan', 'tanh', 'tau', 'trunc
>>> dir()
['__annotations__', '__builtins__', '__doc__', '__loader__', '
 'fun', 'fun1', 'fun2', 'func1', 'func2', 'hi', 'math', 'pi',
>>>
```

Modules

4. Storing Your Functions in Modules

- For example, you define 3 functions add(a,b), fib_prt(n), fib_seq(n),
- and you store these functions in a separate file(fibo.py) called a module,
- and then you or someone else can import that module into the main program.

```
# fibo.py
def add(a, b):
    return a+b
def fib_prt(n):
    a, b = 0, 1
    while b<n:
        print(b, end=' ')
        a, b = b, a+b
    print()
```

```
def fib_seq(n):
    result = [ ]
    a, b = 0, 1
    result.append(0)
    result.append(1)
    while b < n:
        result.append(b)
        a, b = b, a+b
    return result
```

Modules

4. Storing Your Functions in Modules

- 注意：有可能出现“ModuleNotFoundError: No module named 'fibo'”
- 那是因为模块路径不在搜索范围内，没有找到。
- 可以通过sys.path查看搜索路径；
- 通过sys.path.append('模块路径')，添加搜索路径

```
# test.py
import fibo          # Import custom modules
fibo.fib_prt(1000)       # Output: 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
fibo.fib_seq(100)        # Output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
fibo.add(2, 3)           # Output: 5
```

Modules

4. Storing Your Functions in Modules

- In development, developers often add test information to modules, such as:

```
# testmodule.py
def add(a,b):
    return a+b
# for testing
result = add(12, 22)
print("in testmodule.py file, 12+22 = %d" % result)
```

- If this module is imported in other py files, such as:

```
import testmodule as tm
result = tm.add(1, 2)
print(result)
```

```
Output Result:
in testmodule.py file, 12+22 = 34
3
```

■ 测试代码也运行了

雨课堂 Rain Classroom

Modules

4. Storing Your Functions in Modules

```
#testmodule.py
def add(a,b):
    return a+b
if __name__ == '__main__':
    result = add(12,22)                          # for testing
    print("in testmodule.py file,12+22=%d" % result)   # for testing
```

- python为每个模块提供了一个__name__属性
- 当其值为__main__时，表明该模块自身在运行，否则就是被引用。

➢ If this module is imported in other py files, such as:

```
import testmodule as tm
result = tm.add(1, 2)
print(result)
```

输出结果：

3

雨课堂 Rain Classroom

补充：Python中的包

- 为了组织好模块，通常将多个模块放在一个包里面。
- 包是Python模块文件所在的目录，且该目录下必须存在__init__.py文件。

```
package_a
|------__init__.py
|------module_a1.py
|------module_a2.py

package_b
|------__init__.py
|------module_b1.py
|------module_b2.py
main.py
```

- 此时，如果 *main.py* 想要引用 *package_a* 中的模块 *module_a1*，可以使用

```
from package_a import module_a1
或者
import package_a.module_a1
```

补充：Python中的包

- 为了组织好模块，通常将多个模块放在一个包里面。
- 包是Python模块文件所在的目录，且该目录下***必须***存在__init__.py文件。

```
package_a
|------__init__.py
|------module_a1.py
|------module_a2.py

package_b
|------__init__.py
|------module_b1.py
|------module_b2.py
main.py
```

- 在此文件中添加“sys.path.append('../')”这句代码
- 在package_a包中所有模块都增加“import __init__”这句代码



- 如果 *package_a* 包中的 *module_a1* 模块想要引入另一个包 *package_b*，则Python默认是找不到的。

补充：Python包的发布

- 模块编写好之后，可以将其打包成可以安装的压缩包进行发布，以分享给其他开发人员。具体步骤如下：

（1）将要发布的包放在与setup.py文件同级的目录中，结构如下：

```
# 模块包目录itheima
|------setup.py
|------package_a
|       |------__init__.py
|       |------module_a1.py
|       |------module_a2.py
|------package_b
        |------__init__.py
        |------module_b1.py
        |------module_b2.py
```


雨课堂 Rain Classroom

补充：Python包的发布

(2) 编辑setup.py文件，在该文件中指明需要包含哪些py文件(模块)。如下：

```
from distutils.core import setup
setup(
    name = "itheima",      # 模块包目录名称
    version = "1.0",
    description = "itheima belongs to itcast",
    author = "itcast"
    py_modules = ['package_a.module_a1', 'package_a.module_a2',
'package_b.module_b1', 'package_b.module_b2']
)
```


雨课堂 Rain Classroom

补充：Python包的发布

(3) 在当前路径下打开命令行窗口，使用 **build** 命令构建模块，如下

```
>>> python setup.py build
```

- 执行命令后，会在当前目录下新建一个build文件夹，该文件夹下的lib目录中存放了所有待发布模块的备份。

```
# 模块包目录itheima
|------build
|       |------lib
|               |------package_a
|               |       |------__init__.py
|               |       |------module_a1.py
|               |       |------module_a2.py
|               |------package_b
|                       |------__init__.py
|                       |------module_b1.py
|                       |------module_b2.py
|------setup.py
|------package_a
|       |------__init__.py
|       |------module_a1.py
|       |------module_a2.py
|------package_b
        |------__init__.py
        |------module_b1.py
        |------module_b2.py
```

补充：Python包的发布

(4) 使用sdist命令生成用于发布的压缩包，如下：

```
>>> python setup.py sdist
```

- 执行命令后，在当前目录的 dist 目录下生成了最终发布的压缩包“itheima-1.0.tar.gz”
- 此时，当前目录的结构如下：

```
# 模块包目录itheima
|------build
|       |------lib
|               |------package_a
|               |       |------__init__.py
|               |       |------module_a1.py
|               |       |------module_a2.py
|               |------package_b
|                       |------__init__.py
|                       |------module_b1.py
|                       |------module_b2.py
|------dist
|       |------itheima-1.0.tar.gz
|------MANIFEST
|------setup.py
|------package_a
|       |------__init__.py
|       |------module_a1.py
|       |------module_a2.py
|------package_b
        |------__init__.py
        |------module_b1.py
        |------module_b2.py
```

- 发布包
- 文件中包含发布包中的文件列表
- 存放元数据

补充：Python包的安装

- 当程序需要引入一些外部模块时，可以从外部下载并安装。
  - 有些模块提供了自动安装的文件，直接双击就可以。
  - 但大多数模块没有提供这样的安装方式。
- 以刚才介绍的发布的模块为例，介绍如何安装，步骤如下：

（1）找到模块的压缩包，解压。

（2）执行命令“**python setup.py install**”进行安装。

- 可以对安装的目录进行设定
- 如：**python setup.py install --prefix=安装路径**

雨课堂 Rain Classroom

Modules

5. Common modules

- time（时间模块）
- calendar（日历模块）
- datetime（日期时间模块）
- random（随机模块）
- math, cmath（数学模块）

雨课堂 Rain Classroom

Modules

5. Common modules

- help documentation in Python official website: *https://docs.python.org/3/*

```
>>> dir(time)                  # Using built-in function dir
>>> print(random.__all__)      # Using the module's __all__ attribute
>>> help(time)                 # Using build-in class help
>>> print(time.__doc__)        #  Using the module's __doc__ attribute
>>> print(math.__file__)       # Using the module's __file__ attribute
```

- 注意：以上了解模块的途径，不是都能使用
- 有些模块并不包含可以阅读的源代码，可能已经融入到解释器内了，或者可能是C语言程序写成的，等

Common modules

(1) time

- In Python, there are usually 3 ways to represent time:
  - **Timestamp**(时间戳), seconds since 1970.1.1 00:00:00(UTC纪元)
  - **Formatted time string**(格式化的时间字符串), e.g.: 'Mon Nov 19 18:30:00 2018'
  - **struct_time**(时间元组), There are 9 elements, they are:
    - tm_year(年), tm_mon(月), tm_mday(日),
    - tm_hour(小时, 0-23), tm_min(分, 0-59), tm_sec(秒, 0-59),
    - tm_wday(星期, 0-6), tm_yday(一年中的第几天, 1-366),
    - tm_isdst（是否是夏时令）

雨课堂 Rain Classroom

Common modules

(1) time

Functions	Description
time.asctime([tupletime])	接受时间元组并返回一个可读形式的时间字符串
time.clock()	用以浮点数计算的秒数返回当前的CPU时间，用来衡量不同程序的耗时，比time.time()更有用
time.ctime([secs])	作用相当于time.asctime(localtime(secs))，获取当前时间字符串
time.gmtime([secs])	接收时间戳并返回时间元组t
time.localtime([secs])	接收时间戳并返回当地时间的时间元组t
time.mktime(t)	接收时间元组并返回时间戳
time.sleep(secs)	推迟调用线程的运行，secs指秒数
time.strftime(format[, t])	接收时间元组，并返回以可读字符串表示的当地时间，格式由fmt决定
time.strptime(string[, format])	根据fmt格式，把一个时间字符串解析成时间元组
time.time()	返回当前时间的时间戳

Common modules

(1) time

- Example:

```
>>> import time
>>> time.asctime()                          # 输入时间元组，返回时间字符串，缺省返回当前时间
Out: 'Sun Sep 19 15:39:11 2021'

>>> time.localtime()                        # 输入时间戳，返回时间元组，缺省返回当前时间
Out: time.struct_time(tm_year=2021, tm_mon=9, tm_mday=19, tm_hour=15,
tm_min=42, tm_sec=0, tm_wday=6, tm_yday=262, tm_isdst=0)

>>> time.time()                             # 输入空，返回当前时间的时间戳
Out: 1632037680.0593019

>>> time.mktime(time.localtime())           #输入时间元组，返回时间戳
Out: 1632037741.0
```


雨课堂 Rain Classroom

Common modules

(2) calendar

Functions	Description
calendar(year, w=2, l=1, c=6, m=3)	返回一个多行字符串格式的某年年历，3个月一行，间隔距离为c
firstweekday()	返回当前每周起始日期的设置。默认载入calendar模块，该返回值为0
isleap(year)	是闰年返回True，否则为False
leapdays(y1, y2)	返回在y1，y2两年之间的闰年总数
month(theyear, themonth, w=0, l=0)	返回一个多行字符串格式的某年某月的月日历
monthcalendar(year, month)	返回一个二维列表，每个子列表都是某年某月的某个星期的日期值，如果不在该月内的日期，值为0

Common modules

(2) calendar

```
>>> print(calendar.calendar(2018))
```

■ 星期一是默认的每周第一天，星期天是默认的最后一天。更改设置需调用calendar.setfirstweekday()函数。

```
In [23]: print(calendar.calendar(2018))
                                  2018

      January                   February                  March
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
 1  2  3  4  5  6  7                1  2  3  4                1  2  3  4
 8  9 10 11 12 13 14       5  6  7  8  9 10 11       5  6  7  8  9 10 11
15 16 17 18 19 20 21      12 13 14 15 16 17 18      12 13 14 15 16 17 18
22 23 24 25 26 27 28      19 20 21 22 23 24 25      19 20 21 22 23 24 25
29 30 31                  26 27 28                  26 27 28 29 30 31

      April                     May                       June
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                   1          1  2  3  4  5  6                1  2  3
 2  3  4  5  6  7  8       7  8  9 10 11 12 13       4  5  6  7  8  9 10
 9 10 11 12 13 14 15      14 15 16 17 18 19 20      11 12 13 14 15 16 17
16 17 18 19 20 21 22      21 22 23 24 25 26 27      18 19 20 21 22 23 24
23 24 25 26 27 28 29      28 29 30 31               25 26 27 28 29 30
30

       July                    August                  September
Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su      Mo Tu We Th Fr Sa Su
                   1                1  2  3  4  5                1  2
 2  3  4  5  6  7  8       6  7  8  9 10 11 12       3  4  5  6  7  8  9
```


雨课堂 Rain Classroom

Common modules

(2) calendar

```
>>> calendar.firstweekday()          # Output: 0, Monday is the default first day of the week
>>> calendar.setfirstweekday(6)      # Set Sunday as the first day of the week
>>> print(calendar.calendar(2018))   # Reprint 2018 Calendar
```

```
                                  2018

      January                   February                  March
Su Mo Tu We Th Fr Sa      Su Mo Tu We Th Fr Sa      Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6                  1  2  3                  1  2  3
 7  8  9 10 11 12 13       4  5  6  7  8  9 10       4  5  6  7  8  9 10
14 15 16 17 18 19 20      11 12 13 14 15 16 17      11 12 13 14 15 16 17
21 22 23 24 25 26 27      18 19 20 21 22 23 24      18 19 20 21 22 23 24
28 29 30 31               25 26 27 28               25 26 27 28 29 30 31

      April                     May                       June
Su Mo Tu We Th Fr Sa      Su Mo Tu We Th Fr Sa      Su Mo Tu We Th Fr Sa
 1  2  3  4  5  6  7                  1  2  3  4  5                  1  2
 8  9 10 11 12 13 14       6  7  8  9 10 11 12       3  4  5  6  7  8  9
15 16 17 18 19 20 21      13 14 15 16 17 18 19      10 11 12 13 14 15 16
22 23 24 25 26 27 28      20 21 22 23 24 25 26      17 18 19 20 21 22 23
```


雨课堂 Rain Classroom

Common modules

(2) calendar

```
>>> print(calendar.month(2018,11))
```

Output the calendar of a certain month of a certain year

```
In [27]: print(calendar.month(2018,11))
   November 2018
Su Mo Tu We Th Fr Sa
                 1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

```
>>> print(calendar.monthcalendar(2018,11))
```

Output: [[0, 0, 0, 0, 1, 2, 3], [4, 5, 6, 7, 8, 9, 10], [11, 12, 13, 14, 15, 16, 17], [18, 19, 20, 21, 22, 23, 24], [25, 26, 27, 28, 29, 30, 0]]

Common modules

(2) calendar

Functions	Description
monthrange(year, month)	返回两个整数，即该月的第一天星期码和该月的天数
setfirstweekday(weekday)	设置每周的起始星期码（0-6：周一-周日）
timegm(tuple)	接收一个时间元组，返回时间戳。和time.mktime(tuple)功能一样
weekday(year, month, day)	返回给定日期的星期码

```
>>> calendar.monthrange(2018,11)
Out: (3, 30) # Thursday(3)，30 days
>>> calendar.weekday(2018, 11, 21)
Out: 2                  # Wednesday(2)
```


雨课堂 Rain Classroom

Common modules

(3) datetime

- Basic date and time types: The datetime module supplies classes for manipulating dates and times.
- **Available Types**
  - class datetime.date
  - class datetime.datetime
  - class datetime.time
  - class datetime.timedelta
  - class datetime.tzinfo
  - class datetime.timezone

Common modules

(3) datetime

① class datetime.date

Functions	Description
timetuple()	返回一个time的时间格式对象。等价于time.localtime()
today()	返回当前日期date对象。等价于fromtimestamp(time,time())
toordinal()	返回公元公历开始到现在的天数。公元1年1月1日为1
weekday()	返回星期几
year, month, day	返回date对象的年、月、日

雨课堂 Rain Classroom

Common modules

(3) datetime

❶ using class datetime.date, Example:

```
>>> from datetime import date
>>> print(date.today())
Out: 2021-09-19
>>> print(date.timetuple(date.today()))
Out: time.struct_time(tm_year=2021, tm_mon=9, tm_mday=19, tm_hour=0, tm_min=0,
tm_sec=0, tm_wday=6, tm_yday=262, tm_isdst=-1)
>>> print(date.toordinal(date.today()))
Out: 738052          # 返回公元公历开始到现在的天数。公元1年1月1日为1
>>> print(date.weekday(date.today()))
Out: 6               # 返回星期几
```


雨课堂 Rain Classroom

Common modules

(3) datetime

② class datetime.time

Functions	Description
class datetime.time(hour=0, minute=0, second=0, microsecond=0, tzinfo=None, *, fold=0)	构造函数，返回一个time对象。所有参数均为可选
time.dst()	返回时区信息的描述。如果实例没有tzinfo参数则返回空
time.isoformat(timespec='auto')	返回HH:MM:SS[.mmmmm][+HH:MM]格式字符串

- Example

```
>>> from datetime import time
>>> t = time(19, 04, 0)
>>> t.isoformat()
Out: '19:04:00'
```


雨课堂 Rain Classroom

Common modules

(3) datetime

③ class datetime.datetime

>>> from datetime import datetime

Functions	Description
datetime.now()	返回当前日期和时间，其类型是datetime
combine()	根据给定的date，time对象合并后，返回一个对应值的datetime对象
ctime()	返回ctime格式的字符串
date()	返回具有相同year、month、day的date对象
fromtimestamp()	根据时间戳数值，返回一个datetime对象
now()	返回当前时间

雨课堂 Rain Classroom

Common modules

(4) random

- Random numbers can be used in mathematics, games and other fields, and are often embedded in algorithms to improve algorithm efficiency and improve

Functions	Description
random.choice(seq)	从序列的元素中随机的挑选一个元素。
random.randrange([start,]stop [,step])	从指定范围内，按指定step递增的集合中获取一个随机数，step默认为1，如random.randrange(6)，从0-5中随机挑选一个整数
random.random()	随机生成下一个实数，它在[0,1)范围内
random.seed([x])	改变随机数生成器的种子seed。
random.shuffle(list)	将序列的所有元素随机排序
random.uniform(x,y)	随机生成下一个实数，它在[x,y)范围内

Common modules

(5) math, cmath

- Math module provides access to the mathematical functions defined by the C standard.
  - Functions provided by math module, except when explicitly noted otherwise, their return values are all floats.
- for complex numbers.

- Numeric and Mathematical Modules
  - `numbers` — Numeric abstract base classes
  - `math` — Mathematical functions
  - `cmath` — Mathematical functions for complex numbers
  - `decimal` — Decimal fixed point and floating point arithmetic
  - `fractions` — Rational numbers
  - `random` — Generate pseudo-random numbers
  - `statistics` — Mathematical statistics functions

Chapter 4: Functions and Modules

Contents

1. Functions
   1. Defining a Function and Using it
   2. Passing Information to a Function
   3. Python decorators
   4. Lambda expressions
   5. Return Values
2. The scope of a variable
3. Closures and recursive calls to functions
4. Build-in Function
5. Modules
6. **Game example**

雨课堂 Rain Classroom

Poker Dealing Program (Console Version)

- 4 players play cards, and the computer randomly distributes 52 cards (excluding JOKER不含大小王) to the 4 players, and displays the cards of each player on the screen.
- The program works as follows:

```
[9, 49, 37, 18, 3, 12, 15, 33, 24, 40, 8, 35, 27, 4, 20, 41, 17, 51, 11, 43, 44,
7, 36, 46, 25, 29, 21, 2, 28, 10, 34, 31, 39, 6, 13, 48, 32, 50, 45, 22, 47, 5,
38, 1, 14, 26, 23, 16, 0, 30, 42, 19]
牌手1： ♠ 6 ♠ 9 ♠ A ♣ 10 ♣ 4 ♣ A ♥ 2 ♥ 3 ♥ 7 ♦ 2 ♦ 5 ♦ K ♦ Q
牌手2： ♠ 2 ♠ J ♠ K ♠ Q ♣ 5 ♣ 6 ♣ 7 ♣ 8 ♣ J ♣ K ♥ 4 ♥ 5 ♥ A
牌手3： ♠ 4 ♠ 7 ♣ 9 ♣ Q ♥ 9 ♥ J ♥ K ♥ Q ♦ 3 ♦ 8 ♦ 9 ♦ A ♦ J
牌手4： ♠ 10 ♠ 3 ♠ 5 ♠ 8 ♣ 2 ♣ 3 ♥ 10 ♥ 6 ♥ 8 ♦ 10 ♦ 4 ♦ 6 ♦ 7
```

Poker Dealing Program (Console Version)

Analysis

- Number the 52 cards to be dealt in suit order(先给扑克编码，即数字化)
  - ♣ 梅花：0-12，♦ 方片：13-25，♥ 红桃：26-38，♠ 黑桃：39-51
- Storing 52 cards(然后存储)
  - Store the *numbers* of all cards in the *pocker list*
- shuffling cards(接下来操作，即洗牌)
  - Defining the function gen_pocker(n)
  - It randomly generates two position indices(位置索引), swaps the cards of these two positions, and performs 100 such random swaps.
- Dealing poker(最后发牌)
  - Add the shuffled pocker numbers to the list of four players in order.

Poker Dealing Program (Console Version)

key code implementation

- Number the 52 cards to be dealt in suit order(先给扑克编码，即数字化)
  - ♣ 梅花：0-12，♦ 方片：13-25，♥ 红桃：26-38，♠ 黑桃：39-51
- Storing 52 cards(然后存储)
  - Store the *numbers* of all cards in the *pocker list*

```
n = 52
pocker = [i for i in range(n)]
```


雨课堂 Rain Classroom

Poker Dealing Program (Console Version)

key code implementation

- shuffling cards(接下来操作，即洗牌)
  - Defining the function gen_pocker(n)
  - It randomly generates two position indices(位置索引), swaps the cards of these two positions, and performs 100 such random swaps.

```
def gen_pocker(n):
    """随机交换牌的顺序100次，达到洗牌目的"""
    for i in range(100):
        p1=random.randint(0, n-1)    # 前后区间都闭合
        p2=random.randint(0, n-1)
        pocker[p1], pocker[p2] = pocker[p2], pocker[p1]
    return pocker
```


雨课堂 Rain Classroom

Poker Dealing Program (Console Version)

key code implementation

- Dealing poker(最后发牌)
  - Add the shuffled pocker numbers to the list of four players in order.

```
(a, b, c, d)=([ ], [ ], [ ], [ ])          # a, b, c, d四个列表分别存储4个人的牌
for x in range(13):                        # 每人13张牌
    m=x*4
    a.append(pocker[m])
    b.append(pocker[m+1])
    c.append(pocker[m+2])
    d.append(pocker[m+3])
```

Poker Dealing Program (Console Version)

key code implementation

```
def getColor(x):
    """获取牌的花色"""
    color=["♣ ", "♦ ", "♥ ", "♠ "]
    c = x//13
    if c<0 or c>=4:
        return "ERROR!"
    return color[c]
```

```
def getPuk(x):
    return getColor(x)+getValue(x)
```

```
def getValue(x):
    """获取牌的牌面大小"""
    value=x % 13 + 1
    if value==1:
        return 'A'
    elif value>=2 and value<=10:
        return str(value)
    elif value==11:
        return 'J'
    elif value==12:
        return 'Q'
    elif value==13:
        return 'K'
```

Poker Dealing Program (Console Version)

key code implementation

- Output:

```
a.sort()              # 排序
b.sort()
c.sort()
d.sort()
print("牌手1: " + ' '.join(a))
print("牌手2: " + ' '.join(b))
print("牌手3: " + ' '.join(c))
print("牌手4: " + ' '.join(d))
```


雨课堂 Rain Classroom

Poker Dealing Program (Console Version)

key code implementation

- Output result:

```
[9, 49, 37, 18, 3, 12, 15, 33, 24, 40, 8, 35, 27, 4, 20, 41, 17, 51, 11, 43,
44, 7, 36, 46, 25, 29, 21, 2, 28, 10, 34, 31, 39, 6, 13, 48, 32, 50, 45, 22,
47, 5, 38, 1, 14, 26, 23, 16, 0, 30, 42, 19]
牌手1： ♠ 6 ♠ 9 ♠ A ♣ 10 ♣ 4 ♣ A ♥ 2 ♥ 3 ♥ 7 ♦ 2 ♦ 5 ♦ K ♦ Q
牌手2： ♠ 2 ♠ J ♠ K ♠ Q ♣ 5 ♣ 6 ♣ 7 ♣ 8 ♣ J ♣ K ♥ 4 ♥ 5 ♥ A
牌手3： ♠ 4 ♠ 7 ♣ 9 ♣ Q ♥ 9 ♥ J ♥ K ♥ Q ♦ 3 ♦ 8 ♦ 9 ♦ A ♦ J
牌手4： ♠ 10 ♠ 3 ♠ 5 ♠ 8 ♣ 2 ♣ 3 ♥ 10 ♥ 6 ♥ 8 ♦ 10 ♦ 4 ♦ 6 ♦ 7
```

Man-machine versus Tic-Tac-Toe game

Game interface (console version)

玩家你是否先走 (y/n): y

玩家你先走.

```
0 | 1 | 2
---------
3 | 4 | 5
---------
6 | 7 | 8
```

你走哪个位置? (0 - 8): 0

```
X | 1 | 2
---------
3 | 4 | 5
---------
6 | 7 | 8
```

电脑下棋位置... 4

```
X | 1 | 2
---------
3 | O | 5
---------
6 | 7 | 8
```

……略过

电脑下棋位置... 6

```
X | X | O
---------
X | O | 5
---------
O | 7 | 8
```

电脑赢! 按任意键退出游戏.

雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Game Rules

- The game is played in the nine-square grid, if one side is ahead in a certain direction (horizontal, vertical, diagonal) to form 3 pieces, the victory will be won.

X	X	O
X	O	
O		

O	X	
O	X	X
O		

雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

- Number each position on the board as shown:

0	1	2
3	4	5
6	7	8

- The move(落子) in the game is to enter the square position code.
- In the game, the list ***board*** represents the chessboard, which stores the placement information of the player and the computer, and the unplaced place is ***'EMPTY'***.
- Because of man-machine battle, computer intelligence needs to be realized.

Man-machine versus Tic-Tac-Toe game

Analysis

- Checkerboard representation(棋盘表示):

```
# global constant(全局常量)
X = "X"
O = "O"
EMPTY = " "

def new_board():
    board = []
    for i in range(9):
        board.append(EMPTY)        # Initialized to empty
    return board
```

Man-machine versus Tic-Tac-Toe game

Analysis

- Show chessboard(显示棋盘)：

```
def display_board(board):
    board2=board[:]          # 创建副本，函数里修改不影响原来列表board
    for i in range(len(board)):
        if board[i]==EMPTY:
            board2[i]=i      # 空子的地方要显示数字，便于玩家查看空子位置
    print("\t", board2[0], "|", board2[1], "|", board2[2])
    print("\t", "---------")
    print("\t", board2[3], "|", board2[4], "|", board2[5])
    print("\t", "---------")
    print("\t", board2[6], "|", board2[7], "|", board2[8], "\n")
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

0	1	2
3	4	5
6	7	8

- 机器人的简单人工智能策略：

① 如果有一步棋可以让机器人在本轮中获胜，就选那一步走

② 否则，如果有一步棋可以让玩家在本轮获胜，就选那一步走，堵住那个位置。

③ 否则，机器人选择最佳空位置走，最佳空位置就是中间那个，第二好位置是4个角，剩下的都算第三好的位置。

- 程序中定义一个元组BEST_MOVES存储最佳方格位置：

```
# 按优劣顺序排序的下棋位置
BEST_MOVES = (4, 0, 2, 6, 8, 1, 3, 5, 7)    # 最佳下棋位置表
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

- 要想实现人工智能，得实现井字棋的输赢判断。不像五子棋输赢判断那么复杂，井字棋只有8种情况表示一方获胜，即3颗同样的棋子排成一线。

x	x	x
3	4	5
6	7	8

0	1	2
x	x	x
6	7	8

0	1	2
3	4	5
x	x	x

x	1	2
x	4	5
x	7	8

0	x	2
3	x	5
6	x	8

0	1	x
3	4	x
6	7	x

x	1	2
3	x	5
6	7	x

0	1	x
3	x	5
x	7	8

- 定义一个元组，含有8元素，每个元素又是一个元组定义了赢的棋子位置

• 162

```
WAYS_TO_WIN = ( (0, 1, 2), (3, 4, 5), (6, 7, 8), (0, 3, 6), (1, 4, 7), (2, 5, 8), (0, 4, 8), (2, 4, 6) )
```

Man-machine versus Tic-Tac-Toe game

Analysis

- Win or lose judgment:

```
def winner(board):
    """逐一检查可能赢的位置，如果某位置都是一方的落子(不是空)，则这一方就赢了，返回赢的这方的标志"""
    WAYS_TO_WIN = ((0, 1, 2), (3, 4, 5), (6, 7, 8), (0, 3, 6), (1, 4, 7), (2, 5, 8), (0, 4, 8), (2, 4, 6))
    for places in WAYS_TO_WIN:
        if board[places[0]] == board[places[1]] == board[places[2]] != EMPTY:
            winner = board[places[0]]
            return winner       # 返回赢方的标志

    if EMPTY not in board:          # 棋盘没有空位置
        return "TIE"                # "平局和棋，游戏结束"
    return False
```

Man-machine versus Tic-Tac-Toe game

Analysis

- Generate a set of legal move positions

```
def legal_moves(board):
    """产生可以合法走棋位置的序列（也就是还未下过子位置）"""
    moves = [ ]
    for square in range(9):
        if board[square] == EMPTY:
            moves.append(square)
    return moves
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

- player moves

```
def human_move(board):
    """玩家在一个合规的位置落子"""
    legal = legal_moves(board)
    move = ask_number("你走哪个位置? (0 - 8):", 0, 9)
    while move not in legal:
        print("\n此位置已经落过子了")
        move = ask_number("你走哪个位置? (0 - 8):", 0, 9)
    return move
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

- Machine moves (requires artificial intelligence)

```
def computer_move(board, computer, human):
    board = board[:]        # 创建局部变量的棋盘副本，函数中修改该棋盘不影响原来列表board
    BEST_MOVES = (4, 0, 2, 6, 8, 1, 3, 5, 7)      # 最佳下棋位置表
    for move in legal_moves(board):      # 逐一迭代合规位置，如果电脑落子某合规位置能赢，就走那个位置
        board[move] = computer
        if winner(board) == computer:
            print("电脑下棋位置..." , move)
            return move
        board[move] = EMPTY      # 该位置落子不能赢，取消这步走棋方案，继续试探
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

- Machine moves (requires artificial intelligence)

```
def computer_move(board, computer, human):
    board = board[:]        # 创建局部变量的棋盘副本，函数中修改该棋盘不影响原来列表board
    BEST_MOVES = (4, 0, 2, 6, 8, 1, 3, 5, 7)        # 最佳下棋位置表
########## 接上一页程序代码 ##########
    for move in legal_moves(board):        # 逐一迭代合规位置，如果玩家落子该位置能赢，就堵住那个位置
        board[move] = human
        if winner(board) == human:
            print("电脑下棋位置..." ,move)
            return move
        board[move] = EMPTY        # 该位置落子玩家不能赢，不用去堵，取消这步走棋方案，继续试探
```

Man-machine versus Tic-Tac-Toe game

Analysis

```
def pieces():
    """询问谁先走，先走方为X，后走方为O，返回电脑方、玩家的角色代号"""
    go_first = ask_yes_no("玩家你是否先走 (y/n): ")
    if go_first == "y":
        print("\n玩家你先走.")
        human = X
        computer = O
    else:
        print("\n电脑先走.")
        computer = X
        human = O
    return computer, human          # 返回一个元组
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

```
def main():
    computer, human = pieces()      # 确定谁先走，选角色代号
    turn = X                        # 初始化先走代号
    board = new_board()             # 初始化空棋盘序列结构
    display_board(board)            # 显示空棋盘
```

```
玩家你是否先走 (y/n): y

玩家你先走.
         0 | 1 | 2
         ---------
         3 | 4 | 5
         ---------
         6 | 7 | 8


你走那个位置? (0 - 8):
```


雨课堂 Rain Classroom

Man-machine versus Tic-Tac-Toe game

Analysis

```
def main():
    ########## 接上一页程序代码 ##########
    while not winner(board):          # 交互下棋
        if turn == human:
            move = human_move(board)
            board[move] = human
        else:
            move = computer_move(board, computer, human)
            board[move] = computer
        display_board(board)
        turn = next_turn(turn)
```

Man-machine versus Tic-Tac-Toe game

Analysis

```
def main():
    ########## 接上一页程序代码 ##########
    the_winner = winner(board)      # 游戏有结果了
    # 游戏结束输出输赢信息
    if the_winner == computer:
        print("电脑赢!\n")
    elif the_winner == human:
        print("玩家赢!\n")
    elif the_winner == "TIE":       # 平局和棋
        print("平局和棋，游戏结束\n")
```


雨课堂 Rain Classroom

思考题

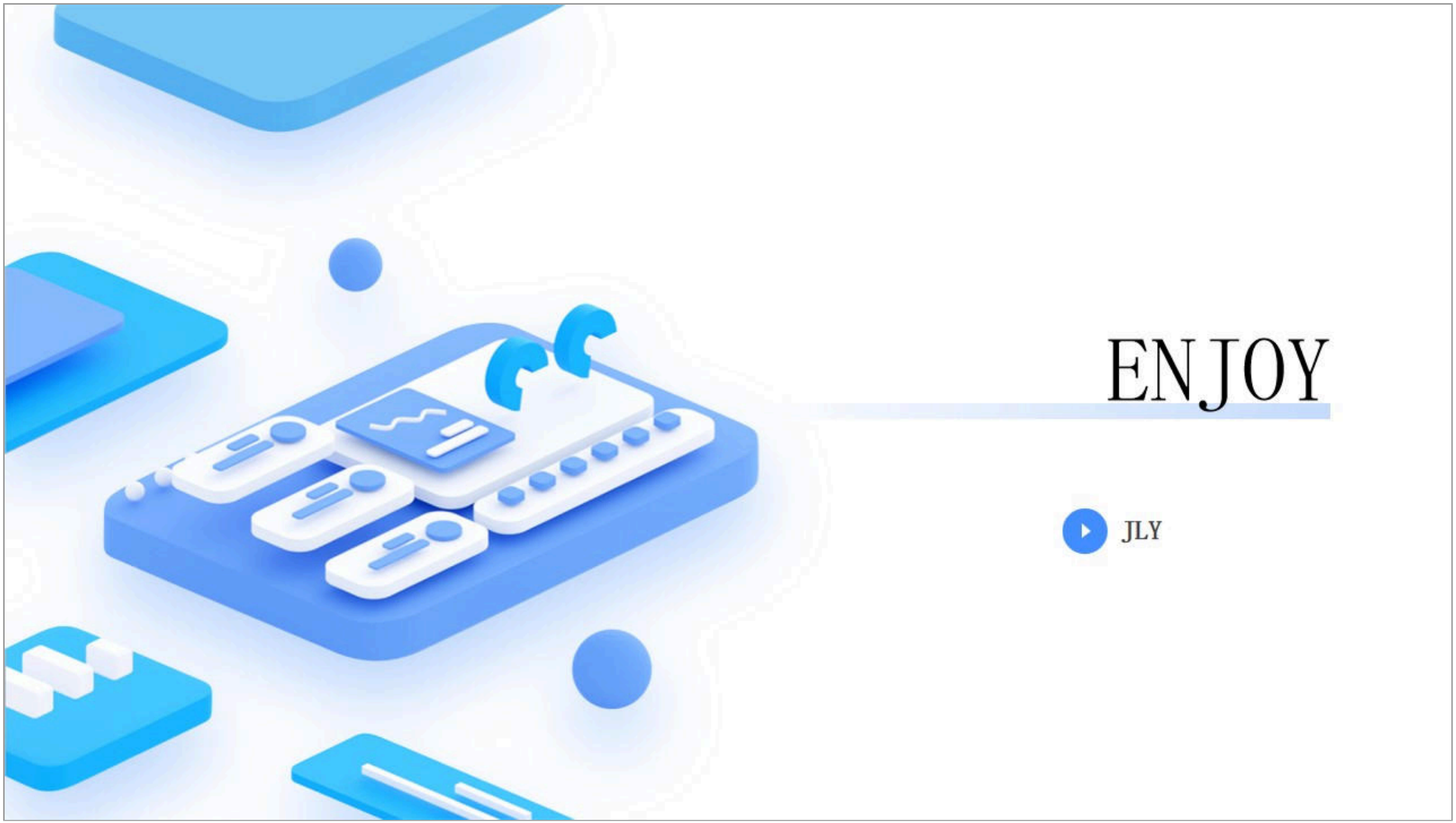
```
swap = lambda a , b : [b , a ]
paws = lambda s : reduce(swap , list (s) , 3)
emit = lambda y : [ x + y for x in y ]
time = lambda t , f : (t and f(t)) or print(t+2)
mite = lambda mu : time( mu+1 , print )
s , t = print , [3*x for x in range (8)]

def ti ( me ):
    while me :
        yield time (me , lambda k : [ k . pop ()])

def newt ( graves ):
    tina = graves . pop ()
    if graves . pop() % 3 == 0:
        graves . pop()
    print ( tina , graves . pop ())
if graves :
    return Pair ( tina -1 , newt ( graves ))
return Pair ( tina +1 , tina )
```

Expression	Interactive Output
5*5	25
print(Pair(6, 1))	(6 . 1)
print(2, s(print(3)))	
emit([[3], [2]])[1]	
s(Pair(Pair(4, 5), nil))	
time(2, mite)	
paws(range(4, 6))	
tuple(ti([5, 6]))	
s(newt([2, 5, 8]))	
s(newt(t))	

雨课堂 Rain Classroom


ENJOY
JLY